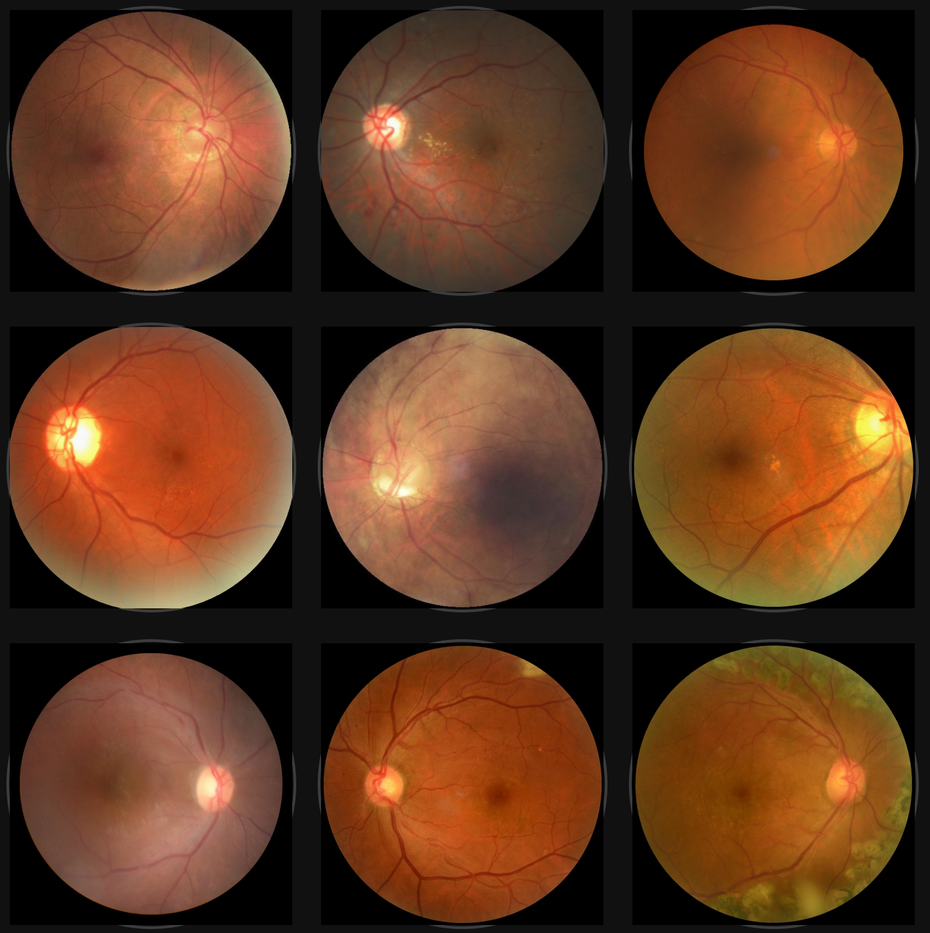
TEAM
EyeQ

SPRINT 1

DUAL

OCULAR DISEASE SCREENER

AI-assisted screening for cataract and glaucoma from a single retinal photograph per eye.



AGENDA

What we'll cover



01 Team roles and responsibilities

02 Improvements from professor feedback

03 Project description

04 Team working agreement

05 Personas

06 MVP

07 Technologies

08 Algorithms

09 Diagrams

10 Product backlog

11 Sprint 1 backlog and test cases

12 Metrics

13 Retrospective

14 Sprint 2 plan

15 Demo and GitHub

01 · TEAM

Roles & responsibilities



Anel Bazarbayeva

Scrum Master, MLOps/ Integration Developer



Negar Kamvij

ML Developer, External Validation & Calibration



Shridhi Patel

Team Leader, ML Developer, Glaucoma Lead



Bhoami Khona

Full Stack Developer



Krushiben Patel

Data Pipeline, MLOps & Integration Developer



Ansh Kevadiya

ML Developer, Cataract Lead



Smit Meghapara

ML Developer, Interpretability & Safety

Improvements from professor feedback

SPRINT 0 FEEDBACK → CHANGES INCORPORATED IN SPRINT 1

01 **Retrospective process**

Use silent individual reflection, group similar ideas, discuss as a team, and agree on the top improvement actions.

02 **Clear action items**

Each retro action fixes one “what can improve” item and has an owner and a check.

03 **Task ownership in planning**

In sprint planning, each member picks their own items in Jira. No work is assigned before the meeting.

04 **Capacity-based planning**

Consider each member’s availability and workload before the team commits to Sprint 2 stories and story points.

05 **Commitments vs. impediments**

Distinguish an unfulfilled commitment from a genuine blocker and define how impediments are communicated and resolved.

Slides changed because of this feedback carry an “Updated after Sprint 0 feedback” tag at the top right.



Project recap

PART 1 OF 4

03 Project description · 04 Working agreement · 05 Personas · 06 MVP
07 Technologies · 08 Algorithms · 09 Diagrams

03 · PROJECT DESCRIPTION

Problem statement

PROBLEM STATEMENT

Cataract and glaucoma cause a large share of the world's vision loss, but they fail patients in opposite ways: cataract is treatable if caught, glaucoma has no symptoms until the damage is already permanent. Both require routine screening that many at-risk patients never get.

EVIDENCE TO CITE

45%

of global blindness caused by cataract (WHO / GBD 2020)

Half

of US glaucoma cases go undiagnosed (CDC)

0.90 → 0.58

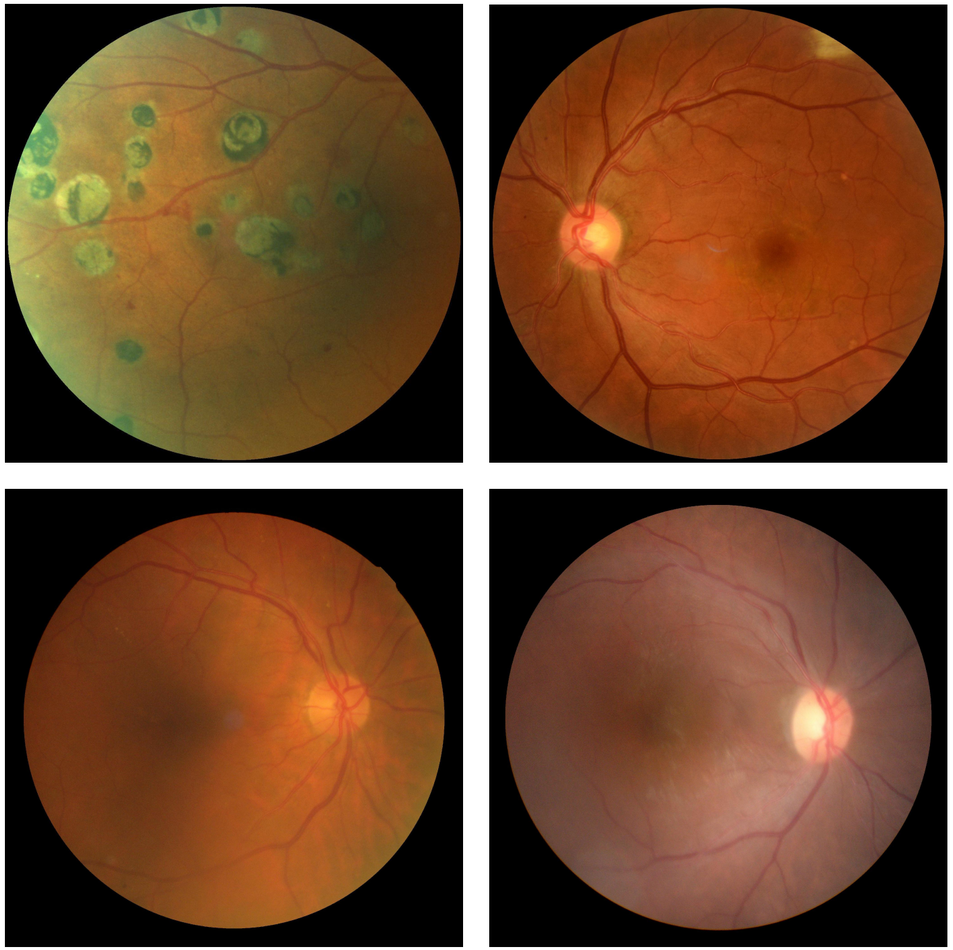
AI screeners: AUC in-house drops across sites (J Clin Med, 2026)

03 · PROJECT DESCRIPTION

Hypothesis

We believe that giving primary care and community screening sites a single-photo AI tool that flags both cataract and glaucoma risk will result in more at-risk patients getting referred before damage is done.

We will know we are right when the model holds up not just on our own test data, but on photos from different fundus cameras and patient populations it never trained on.

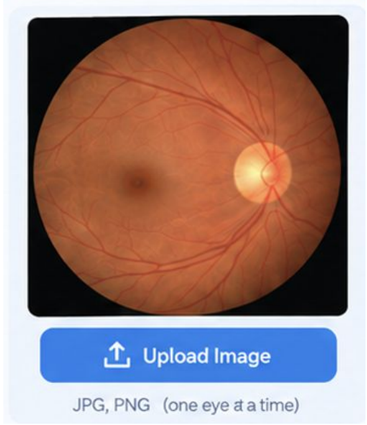


03 · PROJECT DESCRIPTION

What we are building

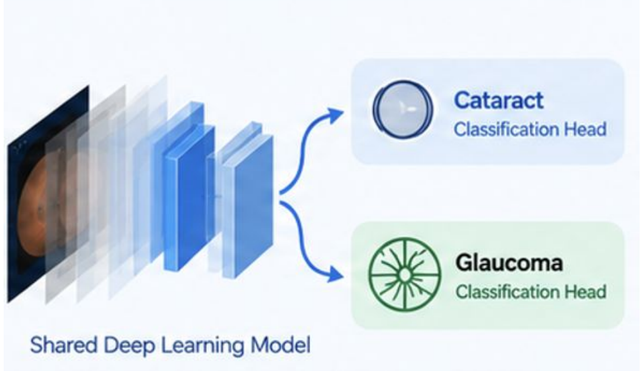
Dual is a web-based AI screening application that analyzes a fundus photograph of the eye and provides separate risk results for **cataract and glaucoma** , with confidence information and a referral recommendation when the image or prediction is uncertain.

01 Upload Fundus Image



One retinal photograph per eye is uploaded to Dual.

02 AI Analysis

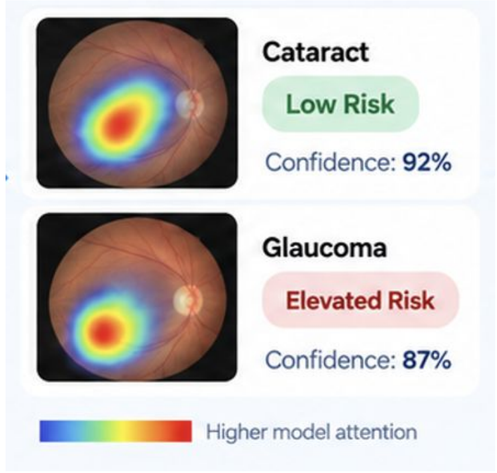


One shared deep learning model analyzes the image for both cataract and glaucoma.

03 · PROJECT DESCRIPTION

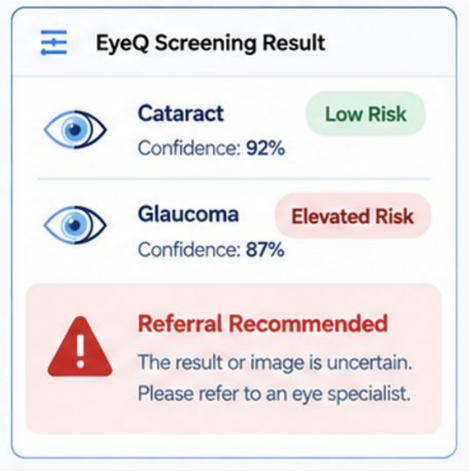
Dual Screening Result

03 Risk Results



Dual provides separate risk results with confidence levels and highlighted regions in the image.

04 Referral Recommendation



Dual provides separate risk results with confidence levels and highlighted regions in the image.

Scope: Dual supports screening and referral decisions. It does not provide medical diagnosis, treatment advice, EHR integration, or other eye conditions.

CLEARER VISION
HEALTHIER LIVES

04 · WORKING AGREEMENT

How we work together

Where & when we meet

Two-hour Zoom meeting every Sunday for project work and planning. Brief follow-up discussions after Tuesday's class.

Communication

For Primary channel, use the team group chat for regular updates and coordination. Respond within 24 hour.

If someone doesn't fulfill their responsibilities

Talk with the member first and work together to resolve the issue. Repeated problems will be discussed as a team and, if necessary, brought to the instructor.

Deadlines & reminders

Set internal deadlines ahead of official due dates. The team leader sends reminders before deadlines. Work is considered complete when it is finished, tested, reviewed, and submitted to the team repository.

04 · WORKING AGREEMENT

Cadence & deliverables

MEETING CADENCE

Standup — Quick progress discussion after class.

Working session — Sunday 2-hour Zoom meeting + individual work throughout the week.

Sprint Duration — Approximately 4 weeks.

SPRINT DELIVERABLES

- **Sprint 0 — SEP-1 to SEP-21**
  Finalize the project scope, team responsibilities, personas, technology stack, datasets, and overall development plan.
- **Sprint 1 — SEP-21 to OCT-19**
  Prepare and preprocess retinal images, create the basic user interface, and build initial cataract and glaucoma model functionality.
- **Sprint 2 — OCT-19 to NOV-16**
  Develop the shared dual-head model, evaluate both disease predictions, integrate the model with the application, and begin external validation.
- **Sprint 3 — NOV-16 to DEC-14**
  Complete the end-to-end application, perform final model testing and validation, improve the interface, and finalize project documentation and presentation



05 · PERSONAS

Helen Whitfield, 68

Retired · At-risk older adult

Last eye exam four years ago

Wants to stay proactive about eye health

GOALS

Identify possible cataract or glaucoma concerns early

Understand her eye health clearly

Know what to do next

NEEDS

Plain-language results

Large, easy-to-read text

A clear next step

CONCERNS

Confusing medical terminology

A frightening result without explanation

Not knowing who to ask for help

> “I may not have symptoms now, but I want to understand my screening result and what I should do next.”

SCENARIO Helen attends a screening visit. A technician captures her fundus image and runs DUAL. She receives a simple explanation of the screening result and the recommended next step. DUAL supports screening and referral; it does not provide a diagnosis.



05 · PERSONAS

Arun Desai, 26

Screening Technician

Works at rural and community health camps

Uses a portable retinal/fundus camera

Focused on efficient, reliable screening

GOALS

Capture high-quality images quickly

Get easy-to-understand screening results

Screen patients efficiently

NEEDS

Immediate image-quality feedback

Simple, step-by-step workflow

Clear next-step guidance

CONCERNS

Poor-quality or ungradable images

Patients leaving before results are ready

Misinterpreting screening results

> “I need clear feedback right away so I can move to the next patient without delays.”

SCENARIO Arun captures a patient’s fundus image at a screening camp and uploads it to the DUAL. The system checks image quality and screens for cataract and glaucoma. Arun receives a clear screening result and recommended next step and can generate a report for clinical review when needed.



“I want a quick, reliable way to know which patients may need specialist review.”

03 | PRIMARY CARE PHYSICIAN PERSONA

05 · PERSONAS

Dr. Maya Rao, 41

Primary Care Physician

Family physician in a busy primary care practice

No ophthalmology specialization

Uses screening information to support referral decisions

GOALS

Identify patients who may need specialist evaluation

Support early detection and preventive care

Review screening information quickly

NEEDS

Clear cataract and glaucoma screening results

Clear risk and uncertainty indicators.

A clear recommended next step

CONCERNS

Missing a patient who may need referral

Over-referring patients unnecessarily

Misinterpreting an uncertain AI result

> “I need a clear screening result that helps me decide when further eye-care evaluation may be appropriate.”

SCENARIO

Dr. Rao reviews a DUAL screening report after a technician captures and analyzes a patient’s fundus image. She checks the cataract and glaucoma screening results, uncertainty, and recommended next step, then decides whether further eye-care evaluation is appropriate. DUAL supports screening and referral; it does not diagnose.

DUAL in Action

Early Detection. A Brighter Tomorrow.

01 CAPTURE



“Time for my eye screening.”

Helen — Patient

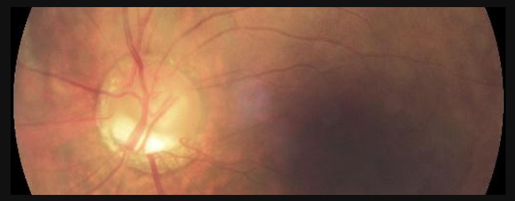


“Let me capture your eye image.”

Arun — Screening Technician

02 SCREEN

DUAL



Checking image...

Screening Cataract + Glaucoma...

Screening complete!

✓ **Cataract**
No significant signs detected

⚠ **Glaucoma**
Possible elevated risk (Refer for further evaluation)

03 REVIEW



“I’ll review the result.”

Dr. Maya — Primary Care Physician

04 NEXT STEP



“Now I know my next step.”

Helen — Patient

06 · MVP

Minimum viable product

One photo. **Two screening results.** **A clear next step.**



Example fundus photo from ODIR-5K

01 **Upload one eye photo**

Choose the patient and left or right eye.

02 **View both screening results**

Separate results for cataract and glaucoma.

03 **See the referral decision**

No concern, Refer, or Uncertain, refer.

Sprint 1

Prototype, trained models
Frozen calibration cutoffs

Sprint 2

Connect the trained model
Run external evaluation

Sprint 3

Grad-CAM heatmaps
Complete app integration

07 · TECHNOLOGIES

Tech Stack

01

Model / ML

Computer vision and deep learning for robust, clinically relevant predictions.

- Python
- PyTorch
- OpenCV
- scikit-learn

02

Backend

High-performance APIs and data services for secure, reliable inference.

- FastAPI
- PostgreSQL

03

Frontend

A responsive, clinician-friendly interface for seamless screening workflows.

- HTML
- CSS
- JavaScript
- React

04

Infra & Tooling

Scalable infrastructure and modern developer tooling for iteration and deployment.

- Docker
- GitHub
- Supabase
- Vite
- Vercel/Render
- Kaggle / Google Colab GPU



TRAINING & VALIDATION DATA

Train: ODIR-5K

External validation / test-only: ORIGA · DRISHTI-GS · ACRIMA · retina_dataset_2016

REAL DATA
STRONGER MODELS
BETTER OUTCOMES

END-TO-END PIPELINE

Fundus image → React UI → FastAPI → PyTorch model → Screening result

FROM IMAGE
TO INSIGHT
FOR BRIGHTER LIVES

08 · ALGORITHMS

Approach & evaluation

CANDIDATE APPROACHES

01 CNN transfer learning
Use ResNet/EfficientNet to learn disease patterns from eye images.

02 Multi-task heads
One model detects both cataract and glaucoma using shared features.

03 Baseline for comparison
Use Logistic Regression as a simple model for comparison.

04 Image-quality classifier
Detects ungradable or poor-quality eye images before prediction

HOW WE WILL JUDGE IT

Sensitivity
How well the model detects actual cataract/glaucoma cases.

Specificity
How well it identifies healthy eyes.

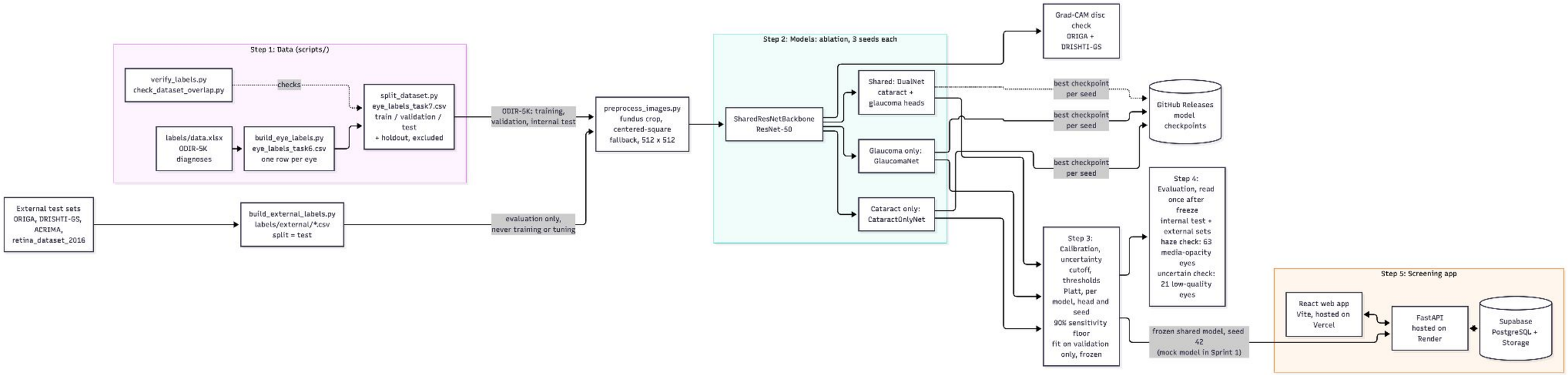
AUC (Area Under the Curve)
Overall ability to distinguish between healthy and diseased eyes.

Confusion Matrix
Shows correct and incorrect predictions for each condition.

Test Set
Evaluate on unseen patient images to measure real-world performance.

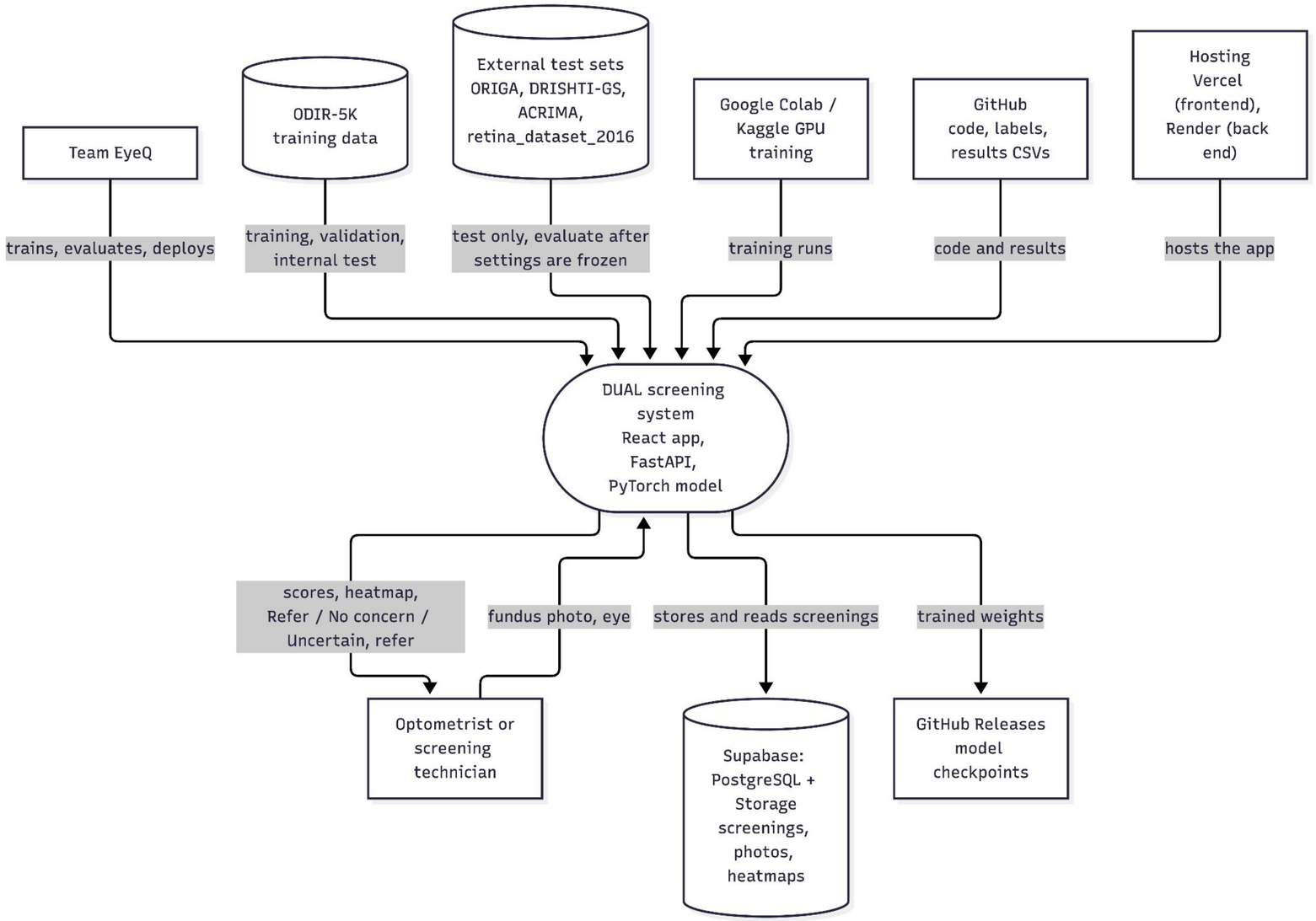
Key goal · Minimize missed disease cases while maintaining reliable predictions.

Architecture (conceptual) Diagram

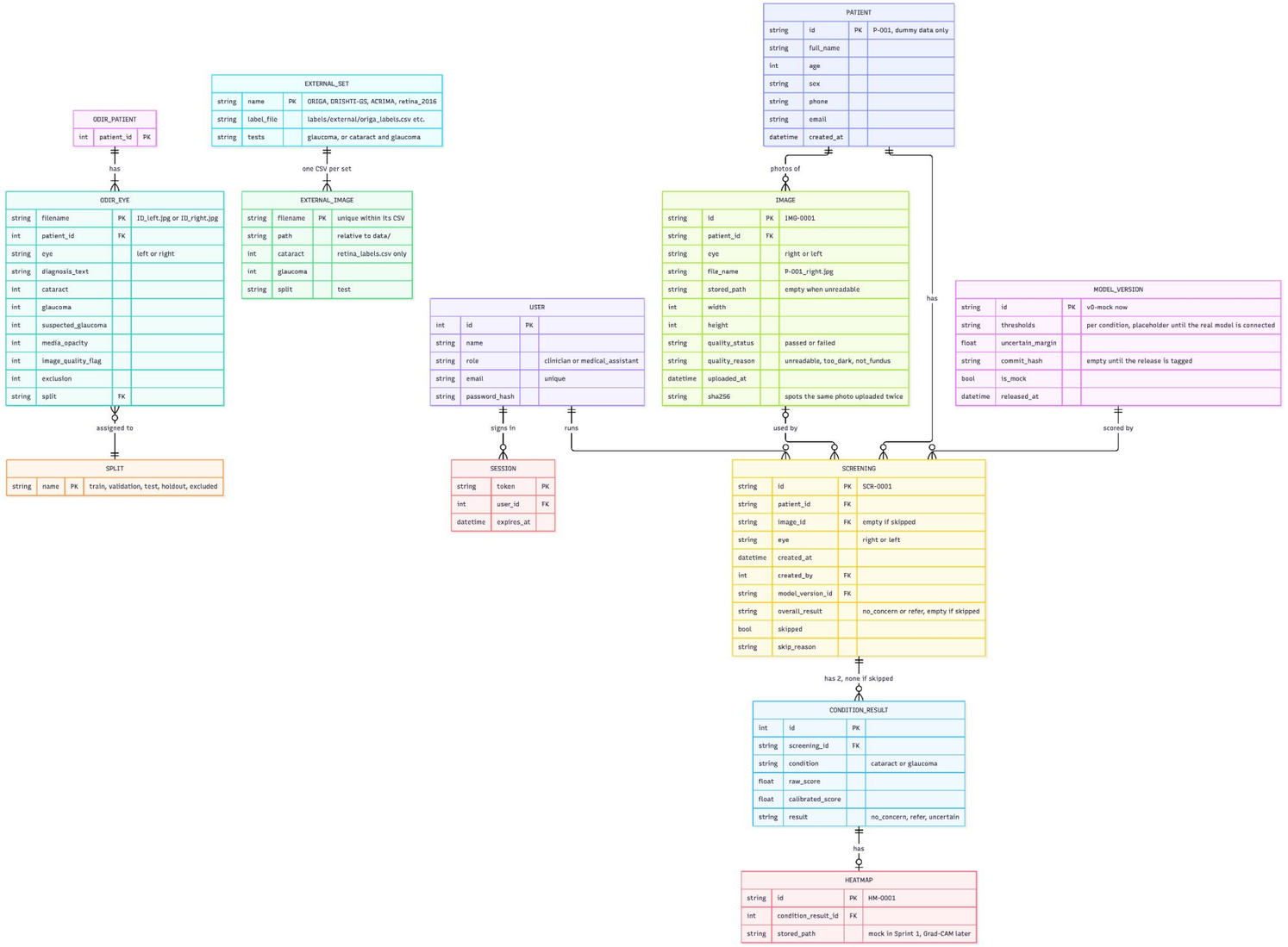


09 · DIAGRAMS

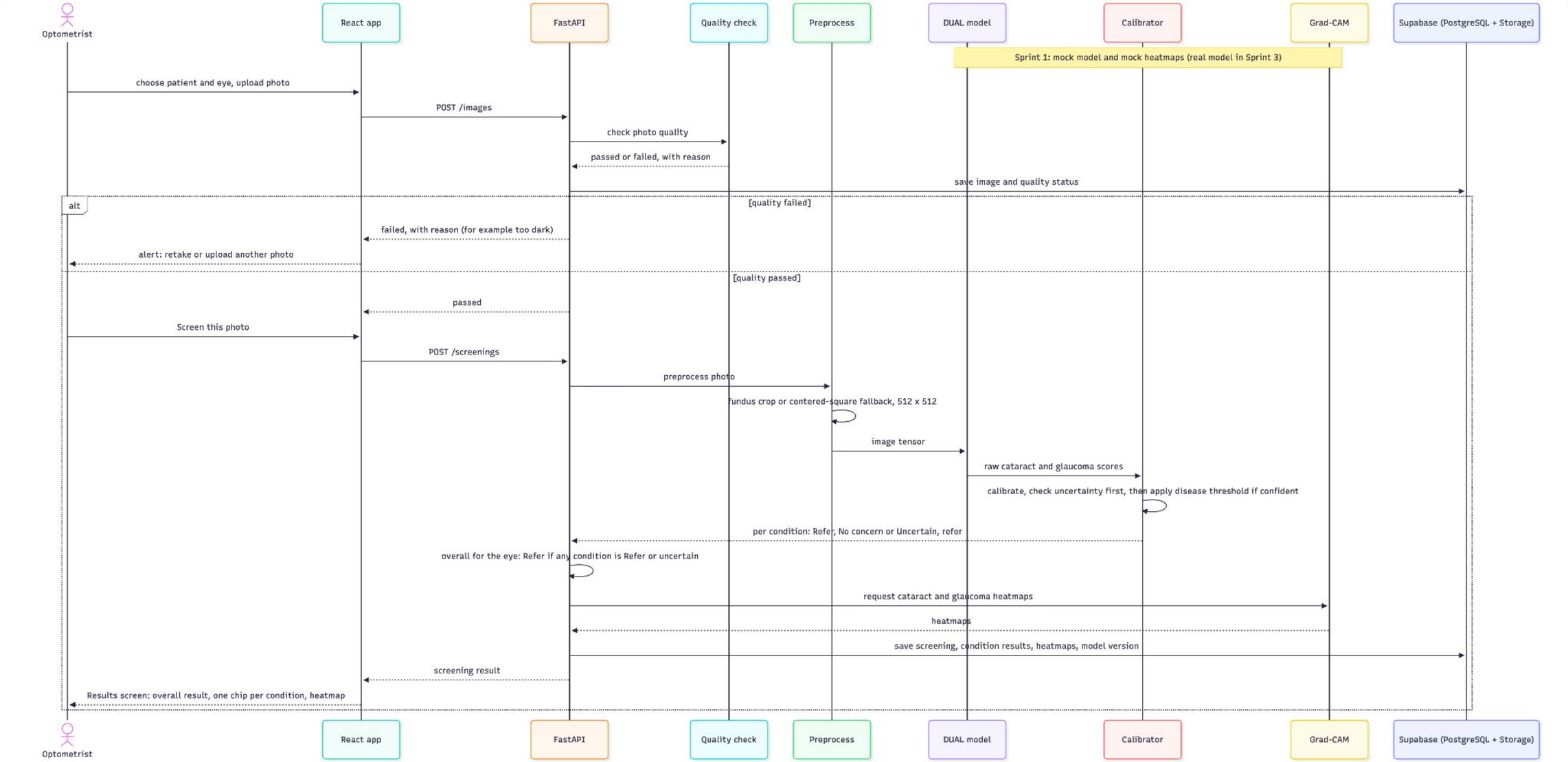
Context Diagram



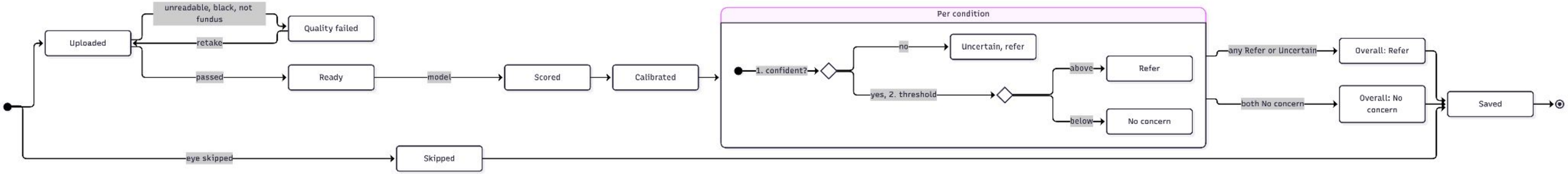
ER Diagram


PATIENT
string id PK P-001, dummy data only
string full_name
int age
string sex
string phone
string email
datetime created_at
photos of
has
EXTERNAL_SET
string name PK ORIGA, DRISHTI-GS, ACRIMA, retina_2016
string label_file labels/external/origa_labels.csv etc.
string tests glaucoma, or cataract and glaucoma
one CSV per set
ODIR_PATIENT
int patient_id PK
has
ODIR_EYE
string filename PK ID_left.jpg or ID_right.jpg
int patient_id FK
string eye left or right
string diagnosis_text
int cataract
int glaucoma
int suspected_glaucoma
int media_opacity
int image_quality_flag
int exclusion
string split FK
assigned to
EXTERNAL_IMAGE
string filename PK unique within its CSV
string path relative to data/
int cataract retina_labels.csv only
int glaucoma
string split test
IMAGE
string id PK IMG-0001
string patient_id FK
string eye right or left
string file_name P-001_right.jpg
string stored_path empty when unreadable
int width
int height
string quality_status passed or failed
string quality_reason unreadable, too_dark, not_fundus
datetime uploaded_at
string sha256 spots the same photo uploaded twice
has
MODEL_VERSION
string id PK v0-mock now
string thresholds per condition, placeholder until the real model is connected
float uncertain_margin
string commit_hash empty until the release is tagged
bool is_mock
datetime released_at
scored by
USER
int id PK
string name
string role clinician or medical_assistant
string email unique
string password_hash
signs in
runs
used by
SESSION
string token PK
int user_id FK
datetime expires_at
SPLIT
string name PK train, validation, test, holdout, excluded
SCREENING
string id PK SCR-0001
string patient_id FK
string image_id FK empty if skipped
string eye right or left
datetime created_at
int created_by FK
string model_version_id FK
string overall_result no_concern or refer, empty if skipped
bool skipped
string skip_reason
has 2, none if skipped
CONDITION_RESULT
int id PK
string screening_id FK
string condition cataract or glaucoma
float raw_score
float calibrated_score
string result no_concern, refer, uncertain
has
HEATMAP
string id PK HM-0001
int condition_result_id FK
string stored_path mock in Sprint 1, Grad-CAM later

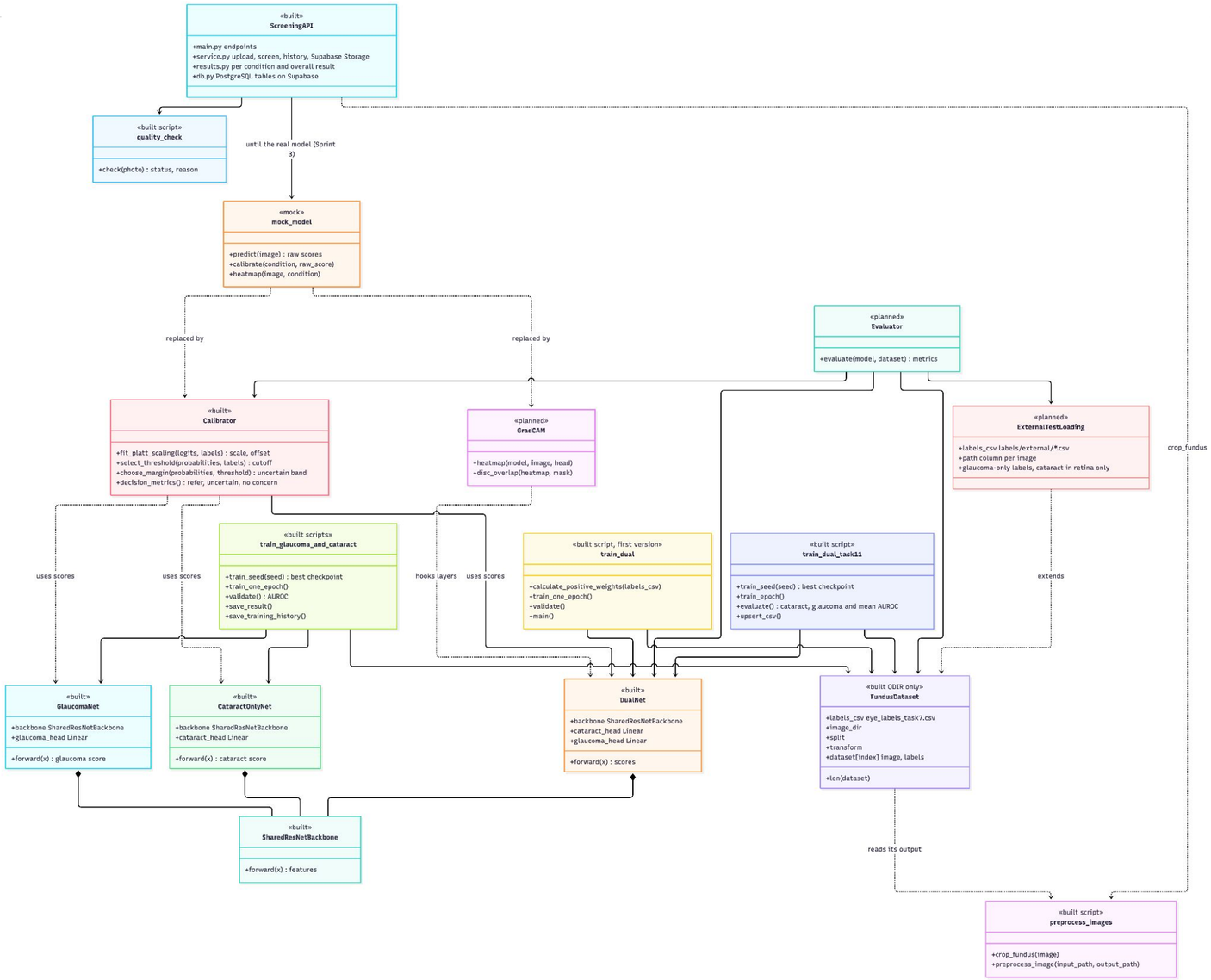
Sequence Diagram



State Diagram



Class Diagram





Sprint 1 work

PART 2 OF 4

10 Product backlog · 11 Sprint 1 backlog, test cases, stories completed

Product backlog – User Stories

ID	Type (F/E)	Feature	User / Technical Story	Acceptance Criteria	Story Points
Sprint 1 (22 Sep to 19 Oct): Data, model and app					**79**
US_01	**E**	Data Pipeline	As the development team, we need clean per-eye labels, a patient-level split and preprocessed photos, so the model can be trained and tested fairly.	Per-eye labels taken from each eye's own keywords pass the verification script; patients are split 70/15/15 with seed 42; one shared preprocessing function runs on every photo; code is merged into main.	21
US_02	**E**	Dual-Head Model	As the ML team, we need one shared ResNet with a cataract head and a glaucoma head trained on the ODIR-5K training split, so that one photo can later give both results in the app.	A shared ResNet with two heads trains end to end on the real labels, weights rare cases, never reads external data, and passes the smoke test.	13
US_03	**E**	External Test Sets	As the ML team, we need test-only labels for four outside datasets and a check that none of their photos are in ODIR-5K, so that the model is tested on data it never saw.	Test-only labels for ORIGA, DRISHTI-GS, ACRIMA and retina_dataset_2016 have checked counts; the overlap check passes; the metrics protocol is written; calibration code is ready and uses the validation split only.	13
US_04	**F**	App Prototype (carried over to Sprint 2)	As a screening technician, I want to upload a patient's fundus photo and see a clear cataract and glaucoma result, so I can decide quickly whether the patient should see an eye specialist.	Upload, Results and History screens work; upload shows a preview and rejects non-images; results show separate cataract and glaucoma results from the API, with "Uncertain, refer" as a normal result; mock data only; follows the DUAL branding.	21
US_05	**F**	Photo Quality Check	As a screening technician, I want the app to stop photos that can't be screened and tell me why, so I don't get a result from an unusable file.	Unreadable, almost-black and non-fundus uploads are blocked with a clear message; hazy or blurry eye photos still go to the model; all ODIR training photos pass; 17 automated tests pass.	3
US_06	**E**	Model Ablation	As the development team, we want cataract-only, glaucoma-only and shared two-head models trained with identical settings, so we can test whether sharing the backbone helps.	Nine models are saved (3 model types x 3 seeds); the training split is used to learn and the validation split to stop; settings for every run are recorded; no external data is used.	8

Product backlog – User Stories

ID	Type (F/E)	Feature	User / Technical Story	Acceptance Criteria	Story Points
Sprint 2 (20 Oct to 16 Nov)					**44**
US_07	**F**	Calibration & Abstention	As an optometrist, I want the tool to say "low image quality or uncertain, refer" when it is not sure, so that I do not act on a weak result.	Calibration, one decision threshold per disease and one abstention cutoff are fitted on the validation split only; the frozen values are committed and not changed afterwards.	5
US_08	**E**	Internal Test	As the development team, we want a within-dataset baseline, so that external results can be compared against it.	All three model types are scored once on the internal test split after calibration is frozen; AUC with 95% confidence interval for each disease; results table is in the repository.	3
US_09	**E**	External Testing	As the ML team, we need the frozen models scored on ORIGA, DRISHTI-GS, ACRIMA and retina_dataset_2016, so that we can report how well the model works on photos from other clinics.	ORIGA and DRISHTI-GS are scored with the frozen thresholds and calibration; ACRIMA and retina_dataset_2016 are reported in their own rows; AUC with confidence intervals and calibration for each set.	5
US_10	**E**	Technical Paper Draft	As the team writing the technical paper, we need the dataset and limitations sections written with verified counts, so that readers can judge the results.	Dataset and limitations sections use counts from the verification script; the cataract external-testing gap is stated as a limitation; PDF and Word versions are on the wiki.	5
US_11	**E**	Cataract Haze Check	As the ML team, we need to check whether the cataract head reacts to blurry or hazy photos rather than the lens, so that a cataract referral can be trusted.	The outcome that means "the model detects haze" is written down before results are seen; cataract share is reported for media-opacity and normal eyes; blur curve is plotted against real cataract eyes.	5
US_16	**E**	Testing and Review	As the team, we want every Sprint 2 story tested and reviewed against its acceptance criteria by someone who did not build it, so that we know each story really works before we close it.	Test cases are written for each Sprint 2 story; each is run and recorded as pass or fail with the date; every pull request for a Sprint 2 story is reviewed before it is merged; problems found are logged in Jira and linked to the story.	5

Product backlog – User Stories

ID	Type (F/E)	Feature	User / Technical Story	Acceptance Criteria	Story Points
US_18	**E**	Cloud Back End	As the back-end developer, I need failed uploads cleaned up and deletions kept in step between the database and storage, so that one failed upload never blocks the app.	A failed upload leaves no leftover file, and the next upload works. Deleting a photo never leaves the database and storage out of sync. The API tests run against a separate test database (never the shared one) and pass, including the PDF exports. Anyone can set up the back end from the repo using the schema, .env.example and README. The docs and diagrams describe Supabase, not SQLite.	8
US_12	**E**	Glaucoma Heatmap Check	As the ML team, we need Grad-CAM heatmaps checked against the optic disc outlines on ORIGA and DRISHTI-GS, so that we can show a glaucoma result is based on the disc.	Grad-CAM heatmaps are produced for ORIGA and DRISHTI-GS; the share of images whose heatmap peak falls inside the disc outline is reported per dataset.	5
US_13	**E**	Ablation Comparison	As the ML team, we need the single-disease and shared models compared per disease, so that the paper can justify one shared backbone.	Single-disease and shared models are compared per disease on the internal test and, for glaucoma, on the external tests; spread across seeds is shown; table and conclusion are in the repository.	3

Product backlog – User Stories

ID	Type (F/E)	Feature	User / Technical Story	Acceptance Criteria	Story Points
Sprint 3 (planned)					34
US_14	F	Real Model in App	As an optometrist, I want to upload a fundus photo and get real cataract and glaucoma risk scores, so that I can decide whether to refer.	The app runs on the trained model; "Uncertain, refer" is shown as a normal result; every results table names the commit that produced it; the results commit is tagged before the final demo.	8
US_15	E	Final Documentation	As a developer joining later, I want install, deployment and API manuals, so that I can run and extend the app.	Data card, user manual, deployment manual and API documentation in PDF and Word; final technical paper with its history page; reviewed by the data pipeline and external validation roles.	8
US_17	E	Testing and Review	As the team, we want every Sprint 3 story tested and reviewed against its acceptance criteria by someone who did not build it, so that the final product we demo and hand over really works.	Test cases are written for each Sprint 3 story; each is run and recorded as pass or fail with the date; the full app is tested end to end with the real model before the final demo; every pull request is reviewed before it is merged; problems found are logged in Jira.	5

Product backlog – User Stories

ID	Type (F/E)	Feature	User / Technical Story	Acceptance Criteria	Story Points
US_19	F	Dashboard and Patients	As a screening technician, I want a dashboard of recent screenings and referrals, a patient directory I can search, and a form to add a new patient, so that I can register patients and find their screenings quickly.	Dashboard shows the number of screenings and referrals and the most recent screenings; patient directory lists every patient, can be searched by name or patient ID, and opens a patient's past screenings; add new patient form checks required fields, saves the patient, and the new patient appears in the directory; data comes from the live back end (API and database changes included in this story); made-up patient data only; pages need sign-in and reload without errors; screens work in both themes and on narrow screens; frontend test cases are written and run.	8
US_20	**E**	MVP Demo	As the course instructor, I want a live demo of the finished MVP working end to end, so that I can check the product against its goals and give final feedback.	Live demo on the deployed app, not screenshots: sign in, add a patient, upload a fundus photo, see both results with the referral advice, and find the screening again in the patient directory; latest test results are shown, including any failures; final model metrics (internal and external) are reported; a backup recording of the same flow is ready in case the live app is down.	5
				Total	**157**

Sprint 1 backlog

ID	Type (F/E)	Feature	User / Technical Story	Acceptance Criteria	Story Points
Sprint 1 (22 Sep to 19 Oct): Data, model and app					**79**
US_01	**E**	Data Pipeline	As the development team, we need clean per-eye labels, a patient-level split and preprocessed photos, so the model can be trained and tested fairly.	Per-eye labels taken from each eye's own keywords pass the verification script; patients are split 70/15/15 with seed 42; one shared preprocessing function runs on every photo; code is merged into main.	21
US_02	**E**	Dual-Head Model	As the ML team, we need one shared ResNet with a cataract head and a glaucoma head trained on the ODIR-5K training split, so that one photo can later give both results in the app.	A shared ResNet with two heads trains end to end on the real labels, weights rare cases, never reads external data, and passes the smoke test.	13
US_03	**E**	External Test Sets	As the ML team, we need test-only labels for four outside datasets and a check that none of their photos are in ODIR-5K, so that the model is tested on data it never saw.	Test-only labels for ORIGA, DRISHTI-GS, ACRIMA and retina_dataset_2016 have checked counts; the overlap check passes; the metrics protocol is written; calibration code is ready and uses the validation split only.	13
US_04	**F**	App Prototype (carried over to Sprint 2)	As a screening technician, I want to upload a patient's fundus photo and see a clear cataract and glaucoma result, so I can decide quickly whether the patient should see an eye specialist.	Upload, Results and History screens work; upload shows a preview and rejects non-images; results show separate cataract and glaucoma results from the API, with "Uncertain, refer" as a normal result; mock data only; follows the DUAL branding.	21
US_05	**F**	Photo Quality Check	As a screening technician, I want the app to stop photos that can't be screened and tell me why, so I don't get a result from an unusable file.	Unreadable, almost-black and non-fundus uploads are blocked with a clear message; hazy or blurry eye photos still go to the model; all ODIR training photos pass; 17 automated tests pass.	3
US_06	**E**	Model Ablation	As the development team, we want cataract-only, glaucoma-only and shared two-head models trained with identical settings, so we can test whether sharing the backbone helps.	Nine models are saved (3 model types x 3 seeds); the training split is used to learn and the validation split to stop; settings for every run are recorded; no external data is used.	8

Backend tests: data preparation

US_01 As the development team, we need clean per-eye labels, a patient-level split and preprocessed photos, so the model can be trained and tested fairly.

TC	TC Title	Step Description	Expected Results	Execution Status	Execution date
TC_01_01	Verification script passes the real label file	1. Clean clone of main. 2. Run: python scripts/verify_labels.py labels/eye_labels_task7.csv --images "data/Internal/ODIR-5K/Training Images" --manifest data	All checks pass	pass	Oct 4
TC_01_02	Verification script reports a wrong file as wrong	1. Run: python scripts/verify_labels.py labels/labels_stub.csv	Stub file fails the count and split checks	pass	Oct 4
TC_01_03	Label file has all columns and 7,000 eyes	1. Run: python scripts/verify_labels.py labels/eye_labels_task7.csv --images "data/Internal/ODIR-5K/Training Images" --manifest data 2. Read the columns and rows lines.	11 columns present; 7,000 rows	pass	Oct 4
TC_01_04	Per-eye disease counts are correct	1. Same run. 2. Read the count lines.	Cataract 313, glaucoma 282, suspected glaucoma 44, media opacity 63, quality flag 426, excluded 30, normal 2,818, diabetic retinopathy 1,796	pass	Oct 4
TC_01_05	One-eye and both-condition patient counts	1. Same run. 2. Read the patient count lines.	Cataract in one eye only 111; glaucoma in one eye only 72; both conditions 1	pass	Oct 4
TC_01_06	Every filename exists and data matches the manifest	1. Same run. 2. Read the filename and MANIFEST lines.	No duplicate filenames; every image found; all 10,324 data files match MANIFEST.sha256	pass	Oct 4
TC_01_07	No patient in two splits	1. Same run. 2. Read the split lines.	No patient in more than one of train, validation, test	pass	Oct 4
TC_01_08	Unusable and held-out eyes kept out of training and testing	1. Same run.	No suspected glaucoma, media opacity or excluded eye in train, validation or test	pass	Oct 4
TC_01_09	Both-condition patient placed in training	1. Same run.	Patient with cataract and glaucoma is in train	pass	Oct 4
TC_01_10	Split shares are 70 / 15 / 15 by patient	1. Same run.	About 70%, 15%, 15% of patients	pass	Oct 4
TC_01_11	Every photo has a preprocessed copy	1. Run: python scripts/preprocess_images.py 2. Count files per dataset in Preprocessed/.	ODIR 7,000; ORIGA 650; DRISHTI-GS 101; ACRIMA 705; retina_dataset_2016 601	pass	Oct 4

WHAT WE CHECK

- Labels match the photos.
- Patients stay in separate splits.
- Preprocessing preserves the original files.

WHY IT MATTERS

The model learns and is tested on correctly prepared data.

Backend tests: external datasets

TC_01_12	Copies are 512 x 512	1. Read the size of every file in Preprocessed/.	Every copy is 512 x 512 pixels	pass	Oct 4
TC_01_13	Originals are not changed	1. After preprocessing, re-run the MANIFEST check (verify_labels.py --manifest data).	All 10,324 original files still match their hashes	pass	Oct 4
TC_01_14	20 random copies look correct by eye	1. Run: python scripts/verify_preprocessing.py 2. Open preprocessing_check.jpg and look at the 20 tiles.	Each tile shows a centred fundus, not cut off or stretched	pass	Oct 4
TC_01_15	Data pipeline scripts run from a clean clone	1. Fresh clone of main (88d55db). 2. Run every test above.	Scripts run without import errors	pass	Oct 4

US_02 As a screening technician, I want one photo to give both a cataract and a glaucoma result, so I can triage both conditions in one step.

TC	TC Title	Step Description	Expected Results	Execution Status	Execution date
TC_02_01	Training smoke test runs end to end	1. Preprocess ODIR (SCRUM-28). 2. Run: python scripts/test_train_dual.py	Training batches and validation batches run with finite loss, no errors	pass	Oct 4

US_03 As a reviewer, I want the model tested on outside datasets it never trained on, so I can trust that it works beyond ODIR-5K.

TC	TC Title	Step Description	Expected Results	Execution Status	Execution date
TC_03_01	ORIGA labels	1. Run: python scripts/build_external_labels.py	650 photos, 168 glaucoma	pass	Oct 4
TC_03_02	DRISHTI-GS labels and spreadsheet cross-check	1. Same run.	101 photos, 70 glaucoma; folder labels agree with the notching spreadsheet	pass	Oct 4
TC_03_03	ACRIMA labels	1. Same run.	705 photos, 396 glaucoma	pass	Oct 4
TC_03_04	retina_dataset_2016 labels	1. Same run.	601 photos, 100 cataract, 101 glaucoma	pass	Oct 4
TC_03_05	Rebuilt label files match the committed ones	1. After the run: git status labels/	No changes to labels/external	pass	Oct 4
TC_03_06	No identical copy of an ODIR photo in retina_dataset_2016	1. Run: python scripts/check_dataset_overlap.py	Matching fingerprints: 0. The check finds identical files only; resized or edited copies are not detected.	pass	Oct 4

WHAT WE CHECK

- All four external datasets have the expected labels and counts.
- The exact-copy check finds no matching files.

WHY IT MATTERS

External data stays separate for later testing.

Backend tests: calibration and cutoffs

TC_03_07	A planted identical copy is caught	1. Copy one ODIR photo, renamed, into a test folder next to a retina photo. 2. Point the script at that folder.	Matching fingerprints: 1	pass	Oct 4
TC_03_08	Calibration tests pass	1. Copy of main (12db1f8). 2. Run: python -m pytest tests/test_calibration_selection.py tests/test_calibration_provenance.py	18 passed	pass	Oct 18
TC_03_09	Calibration uses the validation split only	1. Copy of main (12db1f8). 2. Read every results/calibration/*_validation.csv and results/calibration/shared/*_validation.csv with its .meta.json. 3. Match the filenames against the validation rows of labels/eye_labels_task7.csv.	9 score files (6 single-disease, 3 shared in shared/), 1,024 rows each; filenames match the 1,024 validation eyes exactly; all marked split = validation; no train, test or external eye	pass	Oct 18
TC_03_10	External label files are test-only	1. Copy of develop (0f21e1f, PR #43). 2. Read labels/external/*.csv.	2,057 rows across ORIGA, DRISHTI-GS, ACRIMA and retina_dataset_2016; every row has split = test	pass	Oct 16
TC_03_11	External evaluation protocol is written	1. Copy of develop (0f21e1f, PR #43). 2. Open documents/evaluation/external_validation_protocol.md (SCRUM-44).	Protocol lists the metrics, the freeze rule and the 2,057 test entries; counts match the label files	pass	Oct 16
TC_03_12	Calibration scripts handle empty results and a worse fit	1. Review and run the Task 12 calibration scripts on develop. 2. Check what happens when no threshold candidate is found and when the Platt fit makes the loss worse.	No empty threshold file is written; a worse fit gives a warning (tiny increase) or an error. Issues found on 11 Oct were fixed in PR #38	pass	Oct 12
TC_03_13	Final cutoffs match the approved values	1. Copy of main (12db1f8). 2. Compare results/calibration/frozen_settings.json with the approved CSV (task12_settings_90pct_floor.csv, attached on PR #43).	All 12 model/disease/seed settings (Platt values, cutoff, margin, eyes found, referrals) identical to the approved CSV; the CSV's SHA-256 is recorded in the frozen file	pass	Oct 17
TC_03_14	Settings are frozen before the low-quality check	1. Open results/calibration/frozen_settings.json. 2. Check the commit order on main: freeze 2920a71, then low-quality report 2688f8c.	status = frozen, frozen = true, dated 17 Oct; review reference PR #43; app model = shared seed 42; frozen before the low-quality check ran	pass	Oct 17
TC_03_15	Low-quality photo check uses the frozen settings	1. Open results/calibration/final_holdout_report.md and .json.	Runs once on the 21 low-quality eyes (19 patients) with the frozen settings (SHA matches). Shared seed 42: 8 of 21 Uncertain.	pass	Oct 17

CURRENT RULE: FROZEN

$$t^* = \arg\max_t \ \text{Specificity}(t)$$

$$\text{subject to} \quad \text{Sensitivity}(t) \geq 0.90$$

$$\text{Sensitivity} = \frac{TP}{TP + FN} \qquad \text{Specificity} = \frac{TN}{TN + FP}$$

WEIGHTED YOUDEN: EXPLORATORY

$$J_w(t) = w\,\text{Sensitivity}(t) + (1 - w)\,\text{Specificity}(t) - 1$$

Choose the cutoff t with the largest Jw. Larger w gives sensitivity more weight.

Compares tradeoffs. The frozen rule stays unchanged.

WHAT WE CHECK

- 18 calibration tests pass.
- Only validation images set the cutoffs.
- All 12 settings match the approved values.

FINAL CHECK

Settings froze before the 21-eye holdout check.

Backend tests: API behavior

US_04 As a screening technician, I want to upload a patient's fundus photo and see a clear cataract and glaucoma result, so I can decide quickly whether the patient should see an eye specialist.

TC	TC Title	Step Description	Expected Results	Execution Status	Execution date
TC_04_01	Login is required	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	Without logging in, the patient list returns 401; a wrong password returns 401	pass	Oct 10
TC_04_02	Logout ends the session	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	After logout, the current-user call returns 401	pass	Oct 10
TC_04_03	Demo data loads	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	4 made-up patients sorted by last name; 5 screenings listed newest first	pass	Oct 10
TC_04_04	New patient gets the next ID	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	Next ID is P-005; a patient with an empty name is rejected (422)	pass	Oct 10
TC_04_05	A fundus photo is screened	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	Photo passes the quality check; screening returns a cataract and a glaucoma result, an overall result and both heatmaps; model is marked as mock	pass	Oct 10
TC_04_06	A failed photo is never screened	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	A screenshot fails as not_fundus and cannot be screened (409); a non-image file fails as unreadable	pass	Oct 10
TC_04_07	Photo must match the patient and eye	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	Screening a right-eye photo as the left eye is refused (409)	pass	Oct 10
TC_04_08	The same eye can be screened again	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	Follow-up screenings are all kept and the newest one counts as latest	pass	Oct 10
TC_04_09	The same photo is not screened twice	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	A second upload of the same photo is refused and points to the first screening	pass	Oct 10
TC_04_10	Database allows one screening per photo	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	A second screening of the same photo is blocked by the database itself	pass	Oct 10
TC_04_11	Skipped eye and today's visit	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	Skipping an eye needs a reason; the skipped eye shows in today's visit with no result	pass	Oct 10
TC_04_12	History search	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	Searching 'lopez' finds only that patient's screenings; filtering by patient works	pass	Oct 10
TC_04_13	Exports are PDFs	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	Screening and patient exports download as PDF files	pass	Oct 10

WHAT WE CHECK

- Login and patient records.
- Photo upload and screening.
- History, PDF exports and error responses.

CURRENT STATUS

These passes cover SQLite. Supabase test updates are planned for Sprint 2.

Backend tests: photo quality

TC_04_14	Unknown IDs return 404	1. On branch task-api-mock (PR #35). 2. Run: python -m pytest tests/test_api.py -v	Unknown patient, screening, heatmap and image IDs return 404	pass	Oct 10

US_05 As a screening technician, I want the app to stop photos that can't be screened and tell me why, so I don't get a result from an unusable file.

TC	TC Title	Step Description	Expected Results	Execution Status	Execution date
TC_05_01	Fundus-like photo passes	1. Run: pytest tests/test_quality_check.py -v	passed	pass	Oct 4
TC_05_02	Fundus circle cut off top and bottom passes (ORIGA style)	1. Run: pytest tests/test_quality_check.py -v	passed	pass	Oct 4
TC_05_03	Photo with white border passes	1. Run: pytest tests/test_quality_check.py -v	passed	pass	Oct 4
TC_05_04	Dim, hazy photo passes (cataract look)	1. Run: pytest tests/test_quality_check.py -v	passed	pass	Oct 4
TC_05_05	Blurry photo passes	1. Run: pytest tests/test_quality_check.py -v	passed	pass	Oct 4
TC_05_06	Real ODIR photos pass (incl. washed-out cataract, very dark glaucoma)	1. Run: pytest tests/test_quality_check.py -v	passed	pass	Oct 4
TC_05_07	Unreadable bytes fail	1. Run: pytest tests/test_quality_check.py -v	failed: unreadable	pass	Oct 4
TC_05_08	Empty file fails	1. Run: pytest tests/test_quality_check.py -v	failed: unreadable	pass	Oct 4
TC_05_09	Truncated JPEG fails	1. Run: pytest tests/test_quality_check.py -v	failed: unreadable	pass	Oct 4
TC_05_10	Almost black photo fails	1. Run: pytest tests/test_quality_check.py -v	failed: too dark	pass	Oct 4
TC_05_11	Blank white photo fails	1. Run: pytest tests/test_quality_check.py -v	failed: not fundus	pass	Oct 4
TC_05_12	Blank grey photo fails	1. Run: pytest tests/test_quality_check.py -v	failed: not fundus	pass	Oct 4
TC_05_13	Text document fails	1. Run: pytest tests/test_quality_check.py -v	failed: not fundus	pass	Oct 4
TC_05_14	Random noise fails	1. Run: pytest tests/test_quality_check.py -v	failed: not fundus	pass	Oct 4
TC_05_15	Blue photo fails	1. Run: pytest tests/test_quality_check.py -v	failed: not fundus	pass	Oct 4

WHAT WE CHECK

- Blurry or hazy fundus photos can continue.
- Unreadable, almost-black and non-fundus inputs stop.

WHY IT MATTERS

Haze can be part of the eye condition, so blur alone should not reject a photo.

Backend tests: trained models

TC_05_16	Tight crop without camera border fails	1. Run: pytest tests/test_quality_check.py -v	failed: not fundus	pass	Oct 4
TC_05_17	Every failure reason has a message for the user	1. Run: pytest tests/test_quality_check.py -v	unreadable, too dark, not fundus all have text	pass	Oct 4

US_06 As the development team, we want cataract-only, glaucoma-only and shared two-head models trained with identical settings, so we can test whether sharing the backbone helps.

TC	TC Title	Step Description	Expected Results	Execution Status	Execution date
TC_06_01	Glaucoma-only training smoke test	1. Copy of main (efe7789) with Preprocessed/ODIR. 2. Run: python scripts/test_train_glaucoma.py	Glaucoma Task 11 smoke test passed	pass	Oct 16
TC_06_02	Cataract-only training smoke test	1. Copy of main (efe7789) with Preprocessed/ODIR. 2. Run: python scripts/test_train_cataract.py	Cataract Task 11 smoke test passed	pass	Oct 16
TC_06_03	Shared two-head training pipeline	1. Copy of main (efe7789) with Preprocessed/ODIR. 2. Run test_training_pipeline() from scripts/test_train_dual_task11.py (synthetic data; the full script also needs the 3 shared checkpoints downloaded).	Two-head training, evaluation, mean AUROC, reload and results update all pass	pass	Oct 16
TC_06_04	Nine models are saved	1. Open the GitHub releases task11-cataract-checkpoints, task11-glaucoma-checkpoints and task11-shared-checkpoints.	9 checkpoint files: 3 model types x seeds 42, 43, 44	pass	Oct 16
TC_06_05	Checkpoints match the recorded hashes	1. Compare the SHA-256 column of results/task11/README.md with the release file digests.	9 of 9 match	pass	Oct 16
TC_06_06	Best epoch matches the training history	1. For each model type, compare results/task11/*_results.csv with *_training_history.csv.	Best epoch and AUROC match for all 9 runs (glaucoma history is rounded to 4 decimals); each run stops 5 epochs after its best epoch	pass	Oct 16
TC_06_07	Same settings for all nine runs	1. Read learning_rate, batch_size, max_epochs and patience in the three *_results.csv files.	Learning rate 0.0001, batch 8, max 30 epochs, patience 5 for all 9 runs	pass	Oct 16
TC_06_08	Only the training split is used to learn; no external data	1. Read scripts/fundus_dataset.py, train_glaucoma.py, train_cataract.py and train_dual_task11.py.	Training reads only the train split, validation decides when to stop, images come from Preprocessed/ODIR; no external dataset in any training script	pass	Oct 16

WHAT WE CHECK

- All three training pipelines run.
- Nine checkpoints match their hashes.
- Results match the training history.

WHY IT MATTERS

The model comparison uses recorded settings and reproducible files.

Test cases (frontend)

TC	TC Title	Step Description	Expected Results	Execution Status
TC_04_15	Select patient and eye	Open Upload. Choose a demo patient and select Left or Right.	Selected patient and eye appear before screening.	**Passed**
TC_04_16	Open main pages	Use navigation to open Home, Upload, Results, History and About.	Available pages open with the correct content.	**Passed**
TC_04_17	Upload and preview a photo	Choose a JPG/PNG fundus photo; review its preview before screening.	Selected photo appears in the upload preview.	**Passed**
TC_04_18	Reject unsupported files	Try uploading a non-image file, such as PDF or TXT.	Unsupported file is rejected with clear feedback.	**Passed**
TC_04_19	Reject poor-quality image	Upload a poor-quality fundus photograph and submit for screening.	Quality warning appears; normal screening does not proceed.	**Passed**

Test cases (frontend)

TC	TC Title	Step Description	Expected Results	Execution Status
TC_04_20	Show screening results	Submit an accepted fundus photo and open the Results page.	Overall result and separate cataract/glaucoma results appear.	**Passed**
TC_04_21	Display risk information	Review the condition scores, thresholds, and result details.	Risk information is shown for available mock-model outputs.	**Passed**
TC_04_22	Handle Refer / Uncertain	Review screens for Refer and Uncertain outputs when returned.	Correct label and follow-up guidance are displayed.	**Passed**
TC_04_23	Review screening history	Open History after completing a screening.	Stored screening records can be retrieved and reviewed.	**Passed**
TC_04_24	Switch light/dark theme	Use the theme toggle on the application interface.	Theme changes and content remains readable.	**Passed**
TC_04_25	Check mobile layout	Open Home, Upload, Results and History at phone width.	Content fits without cutoff or unwanted horizontal scrolling.	**Passed**
TC_04_26	Use demo patient records	Review the patient selection and displayed patient details.	Prototype uses fictional patient records, not real patient data.	**Passed**

Stories completed and not completed

Completed 58 story points

ID	Feature	Points	Done
US_01	Data pipeline	21	21 Sep
US_02	Dual-head model	13	29 Sep
US_03	External test sets	13	16 Oct
US_05	Photo quality check	3	30 Sep
US_06	Model ablation	8	14 Oct

All sub-tasks of these stories are closed in Jira.

Not completed, carried over 21 pts

ID	Feature	Points	Still open
US_04	App prototype	21	Results and Log detail screens, upload and results functions, screen tests

Mostly done: 15 of its 20 sub-tasks are closed (React app, navigation, Upload, About and History screens, mock API). It moves to Sprint 2 with its full 21 points.



Metrics and retrospective

PART 3 OF 4

12 Velocity, burndown, completed / committed ratio · 13 Retrospective

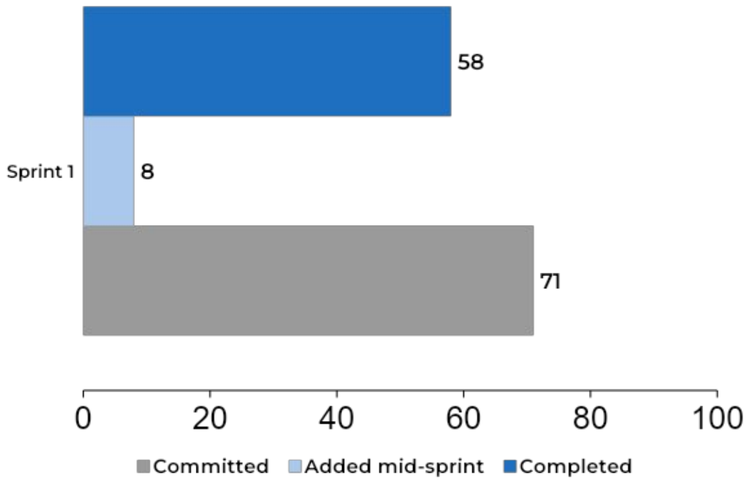
Sprint 1 metrics

Team velocity

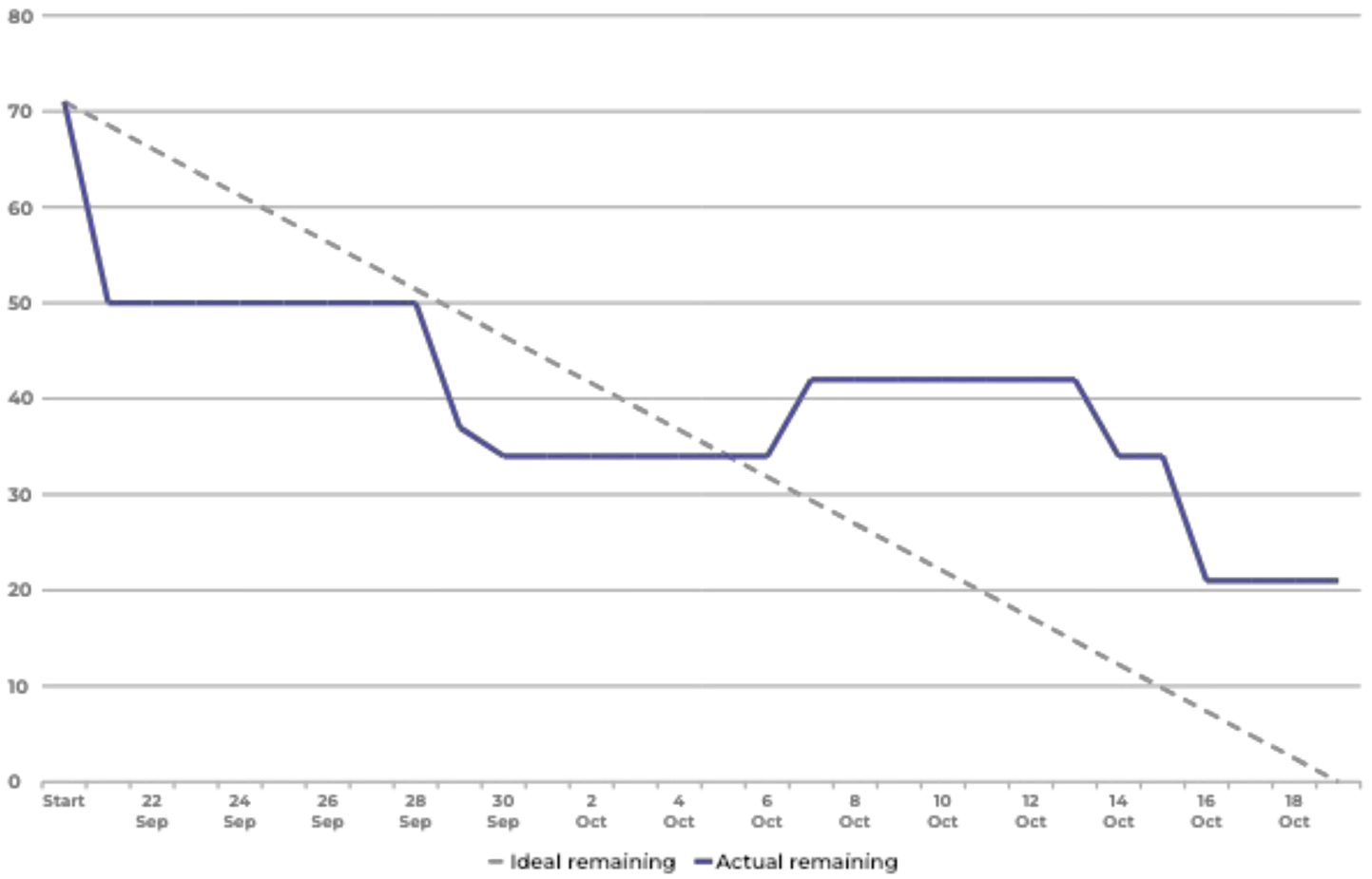
58 story points

Completed in Sprint 1 (story points, not issues)

Velocity chart (story points)



Sprint 1 burndown (story points remaining)



Completed / committed ratio

Sprint 1 ratio

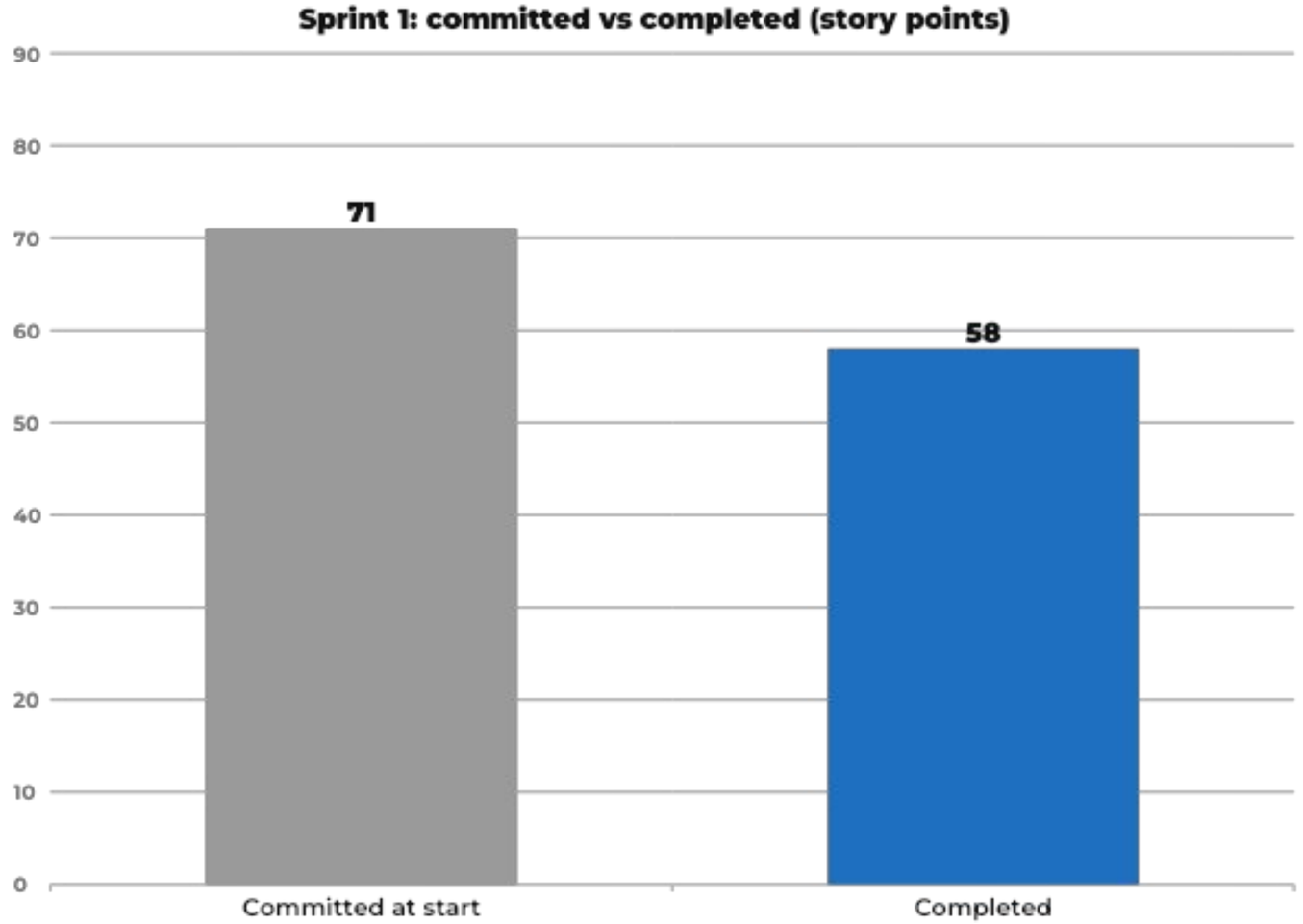
81.7%

58 / 71 story points

Story points completed divided by story points committed at the start (71). US_06 (8 points, added on 7 Oct) counts as completed work but does not raise the commitment. A story counts only when it is finished.

Fibonacci re-estimate (9 Nov): US_01 23 to 21, US_02 16 to 13, US_03 15 to 13, US_04 18 to 21.

Final for Sprint 1 (22 Sep to 19 Oct).



13 · RETROSPECTIVE

Sprint 1 retro

UPDATED AFTER SPRINT 0 FEEDBACK



What went well

- Peer reviews caught real problems before merging.
- Everyone joined the Sunday meetings and stayed engaged.
- We finished the data pipeline, the training code and all nine ablation models.

What can improve

- Code went in without a pull request or a review. Some work was pushed straight to main or develop, and one PR was merged without a submitted review.
- Jira didn’t match the real status. Statuses were updated in bulk, so the Jira burndown was wrong and finished work still showed “To Do”.

ACTION 1 → SPRINT 2

Fixes: code going in without a pull request or review

All changes go through a pull request into develop, with a named reviewer within 48 hours. No direct pushes to main or develop. A PR open longer is raised at the Sunday meeting.

Owner: Whole team

Check: no direct pushes; no PR waits more than 48 hours

ACTION 2 → SPRINT 2

Fixes: Jira not matching the real status

Each member moves their Jira items the same day their work changes status. Done means the CLOSED column. We check the board together at the Sunday meeting.

Owner: Whole team

Check: the Jira burndown matches GitHub; no finished work left in To Do



Sprint 2 plan and demo

PART 4 OF 4

14 Sprint 2 stories and capacity · 15 App, API, GitHub and wiki

Stories planned for Sprint 2

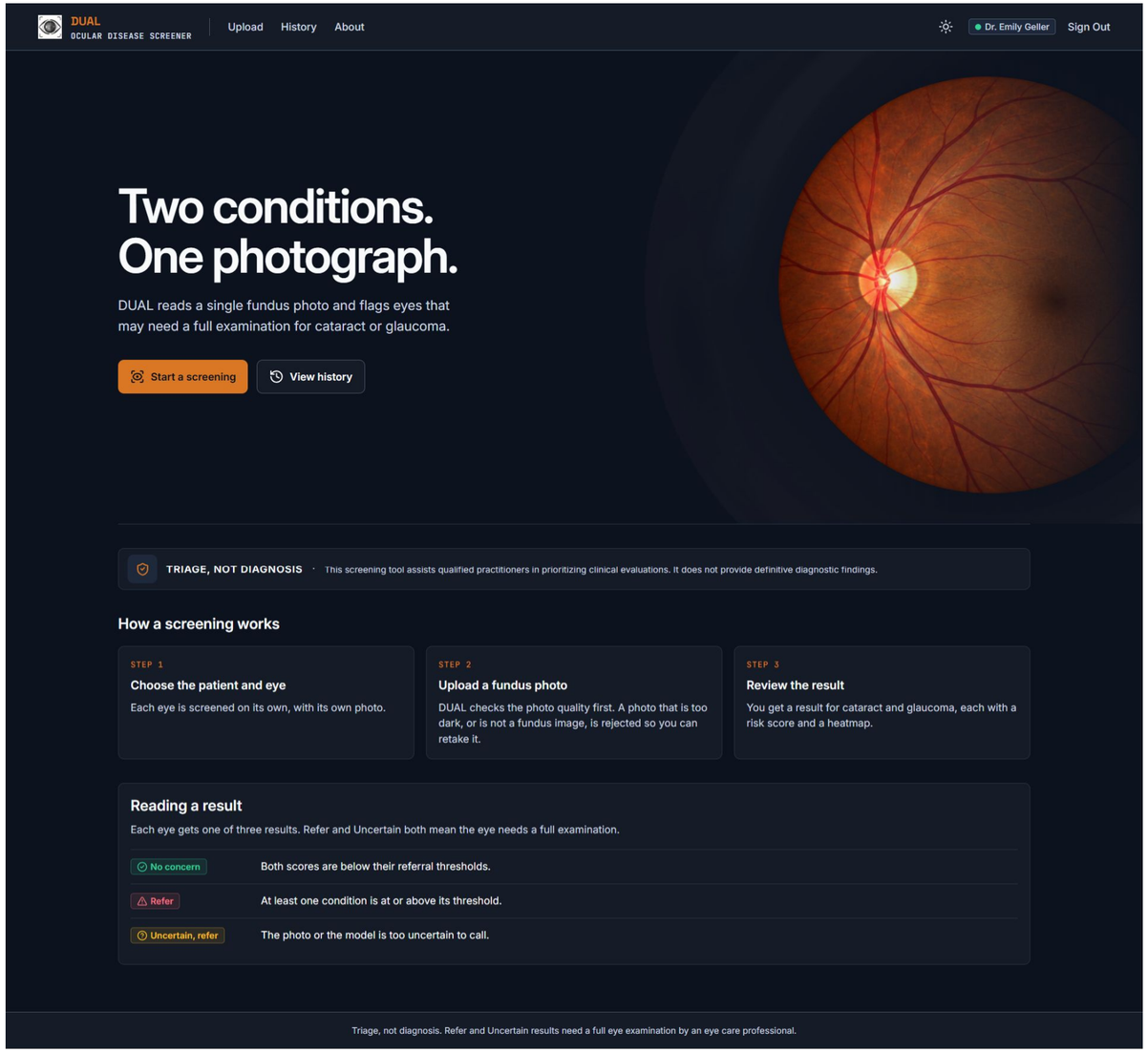
ID	Type (F/E)	Feature	User / Technical Story	Acceptance Criteria	Story Points
Sprint 2 (20 Oct to 16 Nov)					**44**
US_07	**F**	Calibration & Abstention	As an optometrist, I want the tool to say "low image quality or uncertain, refer" when it is not sure, so that I do not act on a weak result.	Calibration, one decision threshold per disease and one abstention cutoff are fitted on the validation split only; the frozen values are committed and not changed afterwards.	5
US_08	**E**	Internal Test	As the development team, we want a within-dataset baseline, so that external results can be compared against it.	All three model types are scored once on the internal test split after calibration is frozen; AUC with 95% confidence interval for each disease; results table is in the repository.	3
US_09	**E**	External Testing	As the ML team, we need the frozen models scored on ORIGA, DRISHTI-GS, ACRIMA and retina_dataset_2016, so that we can report how well the model works on photos from other clinics.	ORIGA and DRISHTI-GS are scored with the frozen thresholds and calibration; ACRIMA and retina_dataset_2016 are reported in their own rows; AUC with confidence intervals and calibration for each set.	5
US_10	**E**	Technical Paper Draft	As the team writing the technical paper, we need the dataset and limitations sections written with verified counts, so that readers can judge the results.	Dataset and limitations sections use counts from the verification script; the cataract external-testing gap is stated as a limitation; PDF and Word versions are on the wiki	5
US_11	**E**	Cataract Haze Check	As the ML team, we need to check whether the cataract head reacts to blurry or hazy photos rather than the lens, so that a cataract referral can be trusted.	The outcome that means "the model detects haze" is written down before results are seen; cataract share is reported for media-opacity and normal eyes; blur curve is plotted against real cataract eyes.	5
US_16	**E**	Testing and Review	As the team, we want every Sprint 2 story tested and reviewed against its acceptance criteria by someone who did not build it, so that we know each story really works before we close it.	Test cases are written for each Sprint 2 story; each is run and recorded as pass or fail with the date; every pull request for a Sprint 2 story is reviewed before it is merged; problems found are logged in Jira and linked to the story.	5

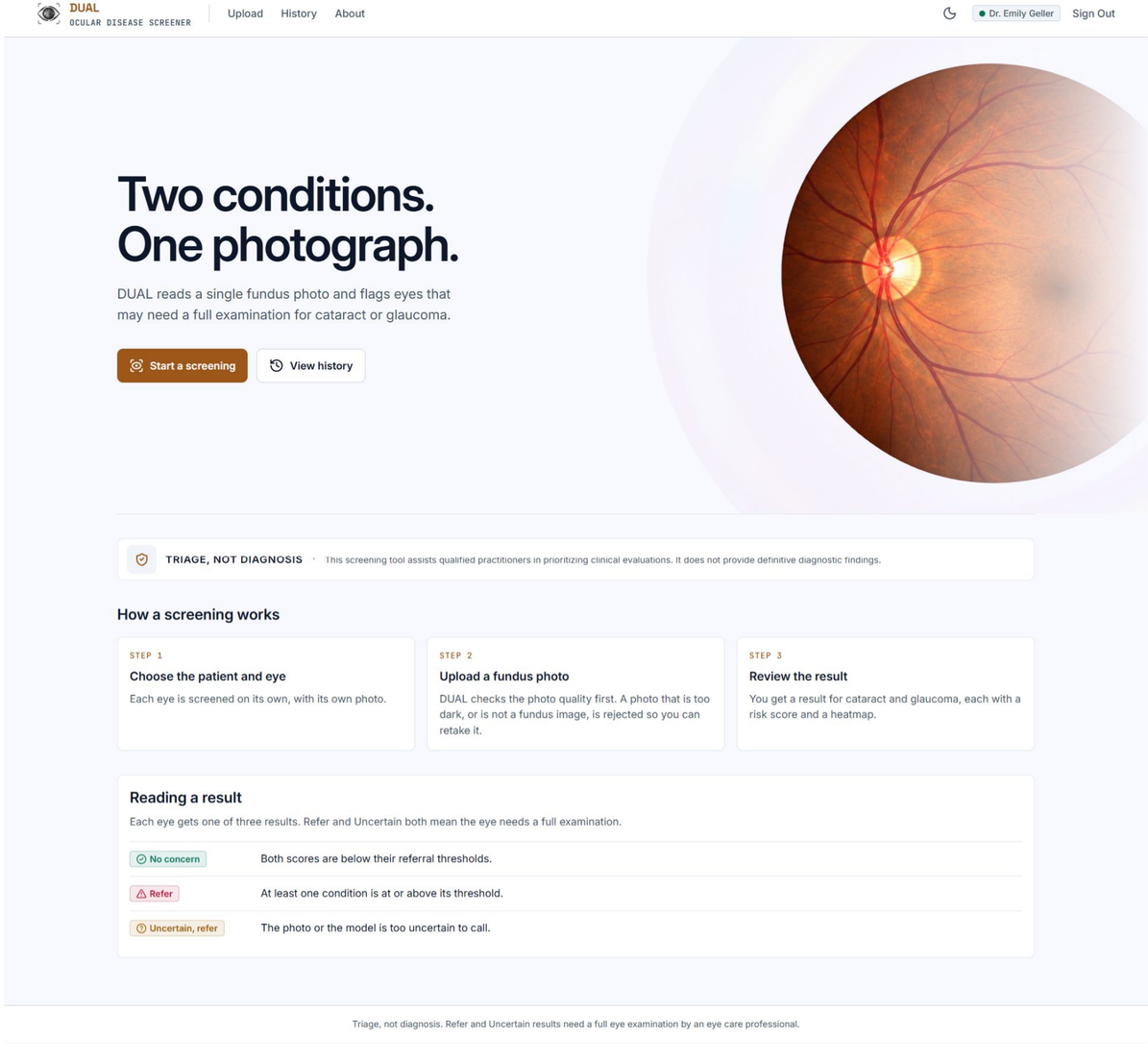
Stories planned for Sprint 2

ID	Type (F/E)	Feature	User / Technical Story	Acceptance Criteria	Story Points
US_18	**E**	Cloud Back End	As the back-end developer, I need failed uploads cleaned up and deletions kept in step between the database and storage, so that one failed upload never blocks the app.	A failed upload leaves no leftover file, and the next upload works. Deleting a photo never leaves the database and storage out of sync. The API tests run against a separate test database (never the shared one) and pass, including the PDF exports. Anyone can set up the back end from the repo using the schema, .env.example and README. The docs and diagrams describe Supabase, not SQLite.	8
US_12	**E**	Glaucoma Heatmap Check	As the ML team, we need Grad-CAM heatmaps checked against the optic disc outlines on ORIGA and DRISHTI-GS, so that we can show a glaucoma result is based on the disc.	Grad-CAM heatmaps are produced for ORIGA and DRISHTI-GS; the share of images whose heatmap peak falls inside the disc outline is reported per dataset.	5
US_13	**E**	Ablation Comparison	As the ML team, we need the single-disease and shared models compared per disease, so that the paper can justify one shared backbone.	Single-disease and shared models are compared per disease on the internal test and, for glaucoma, on the external tests; spread across seeds is shown; table and conclusion are in the repository.	3

15 • DEMO

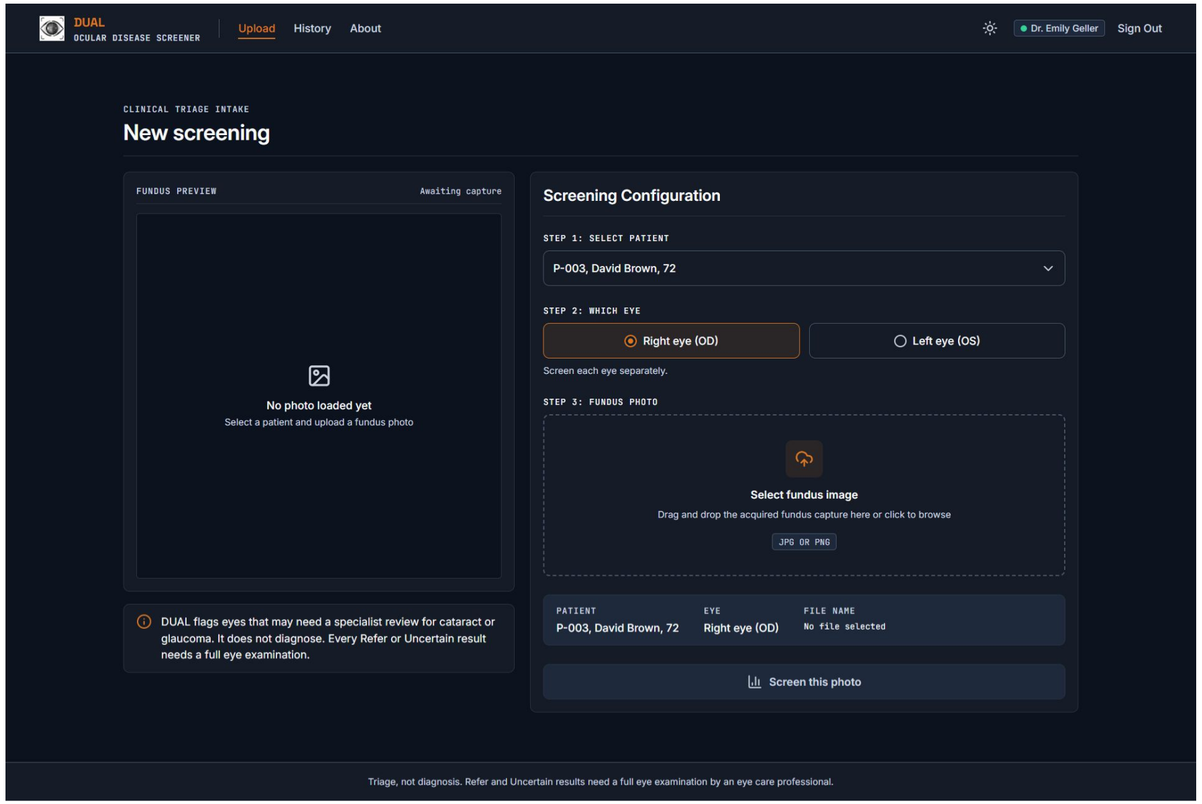
App screenshots: Home screen

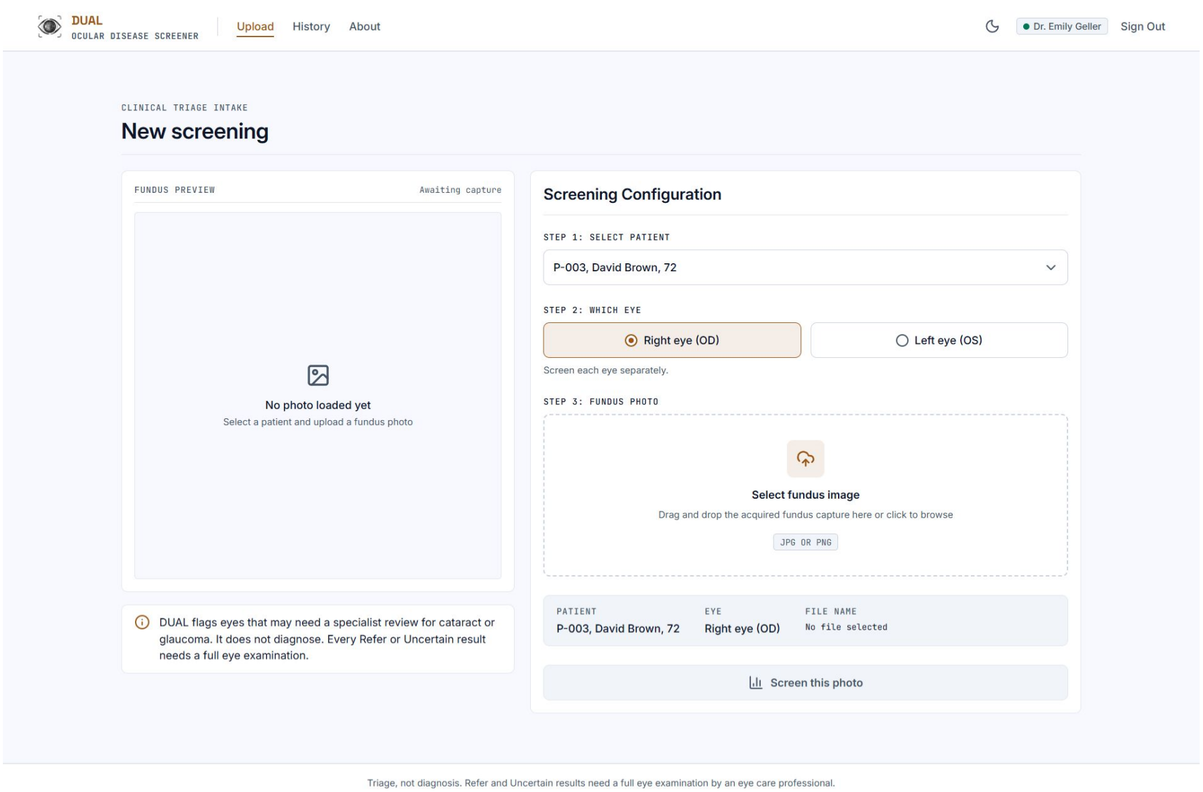




15 • DEMO

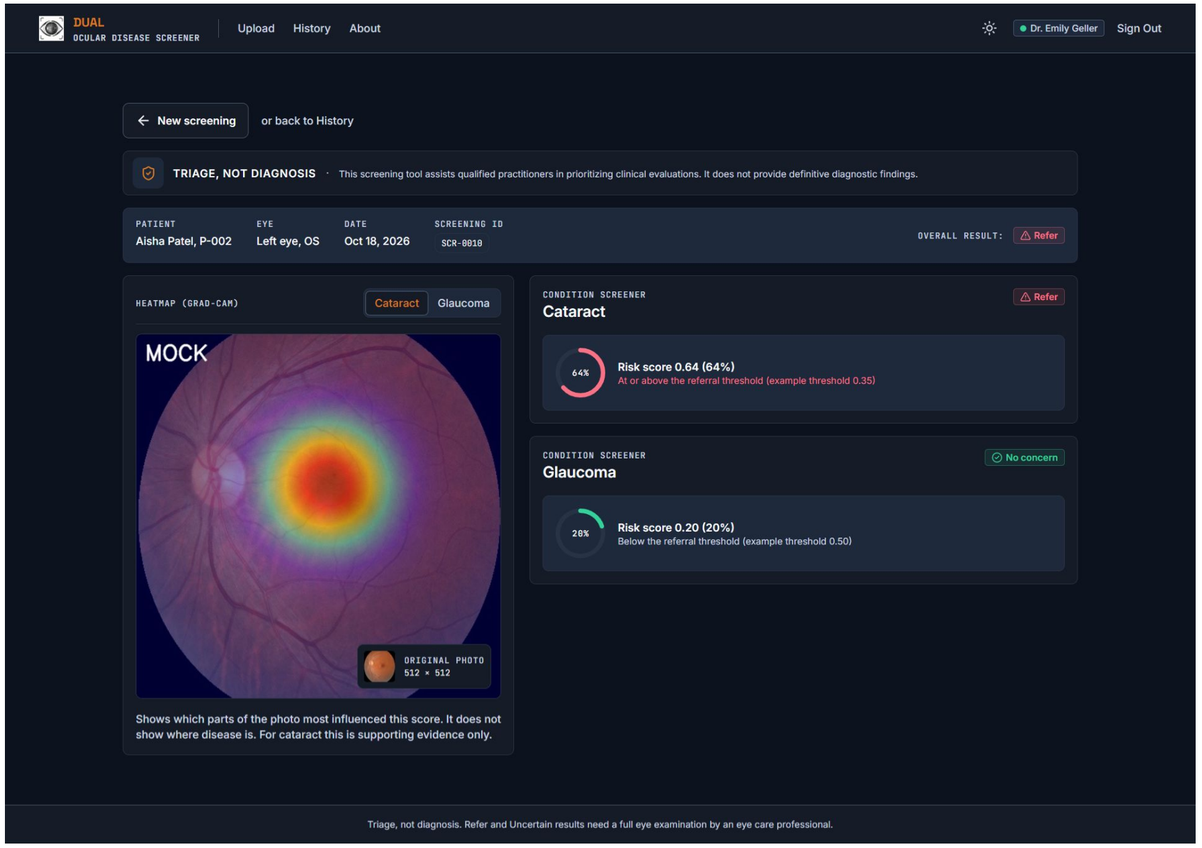
App screenshots: Upload screen

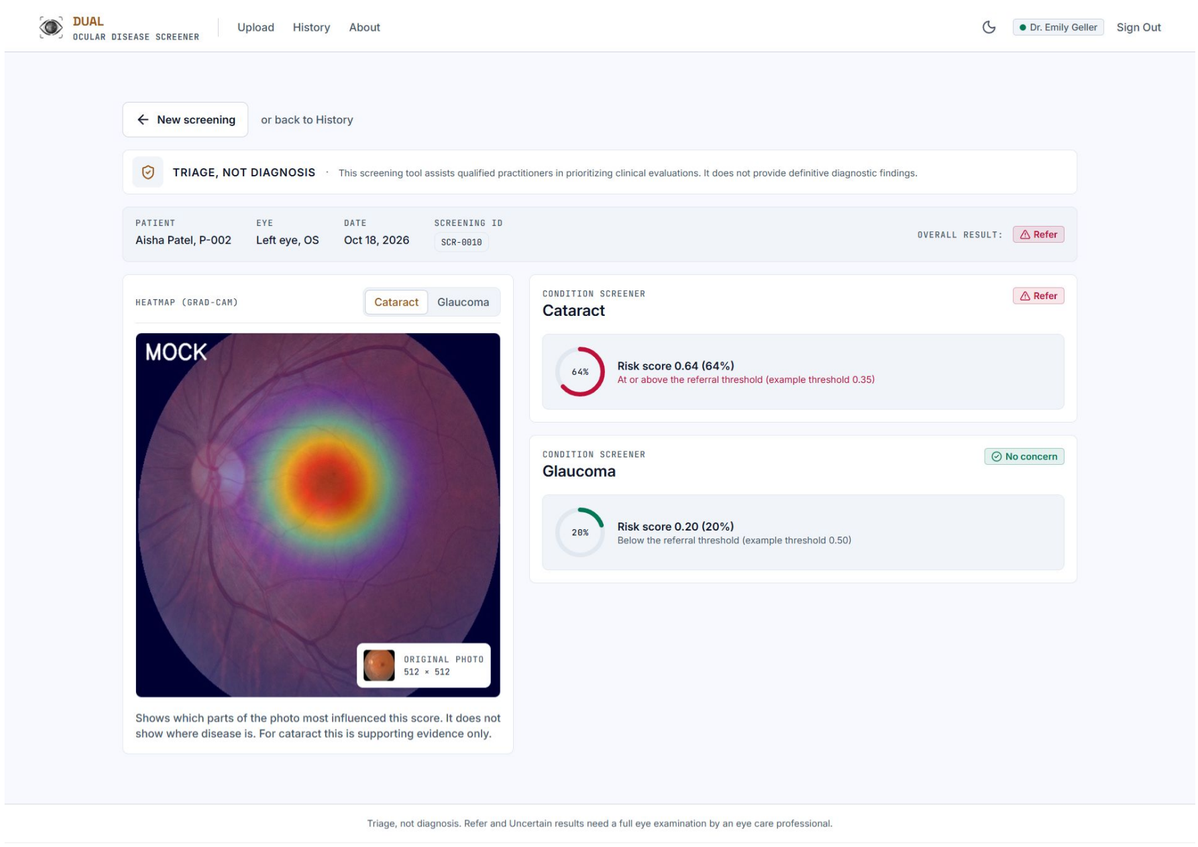




15 • DEMO

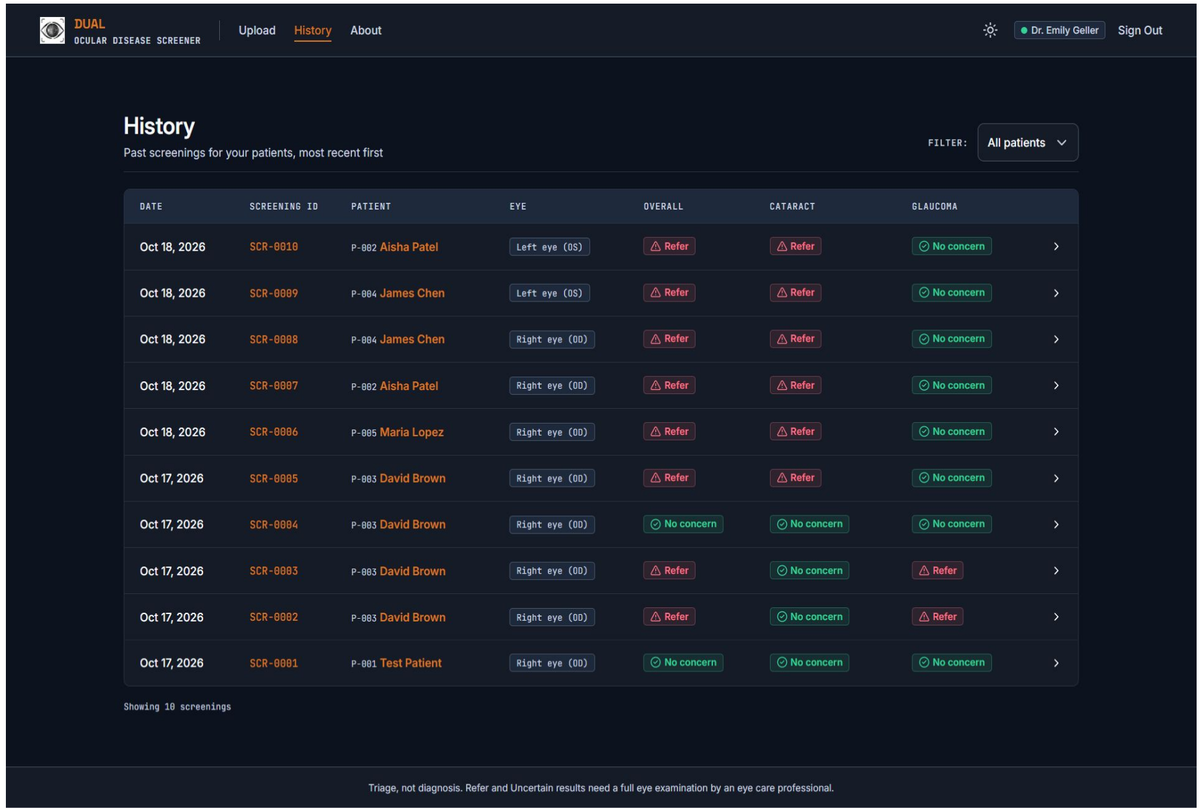
App screenshots: Result screen

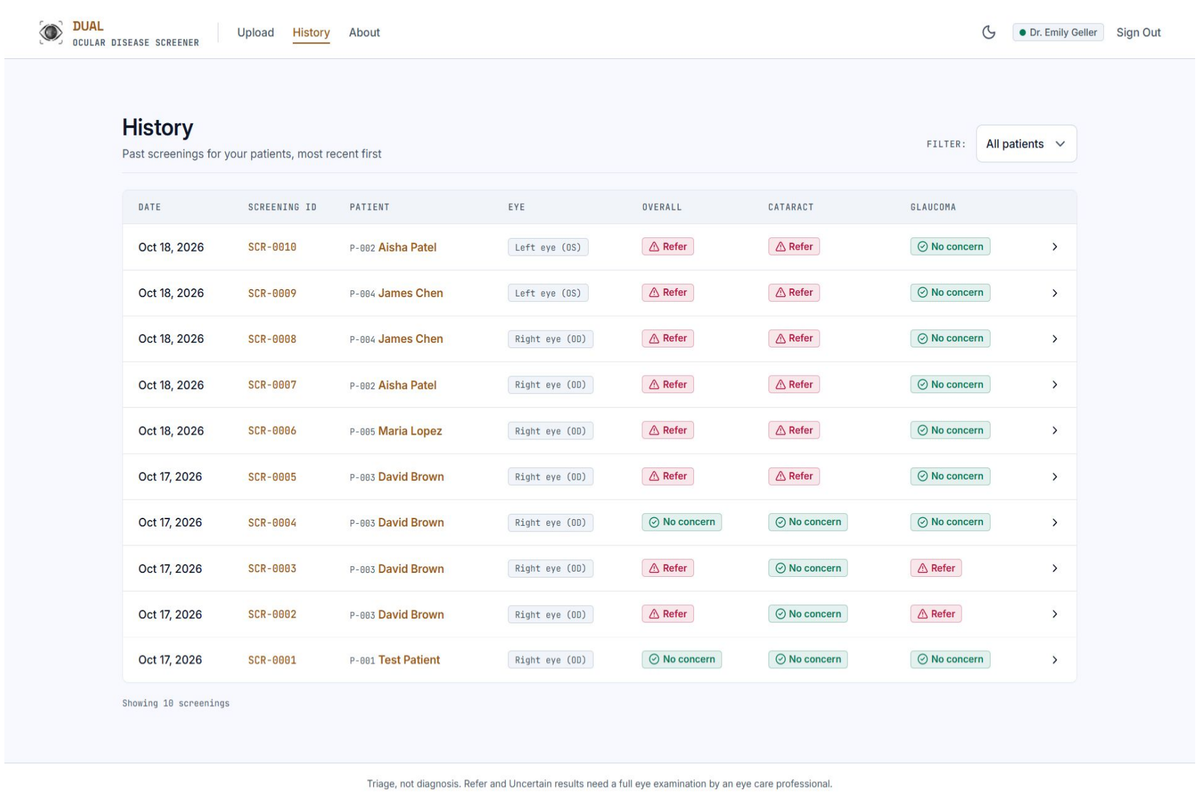




15 • DEMO

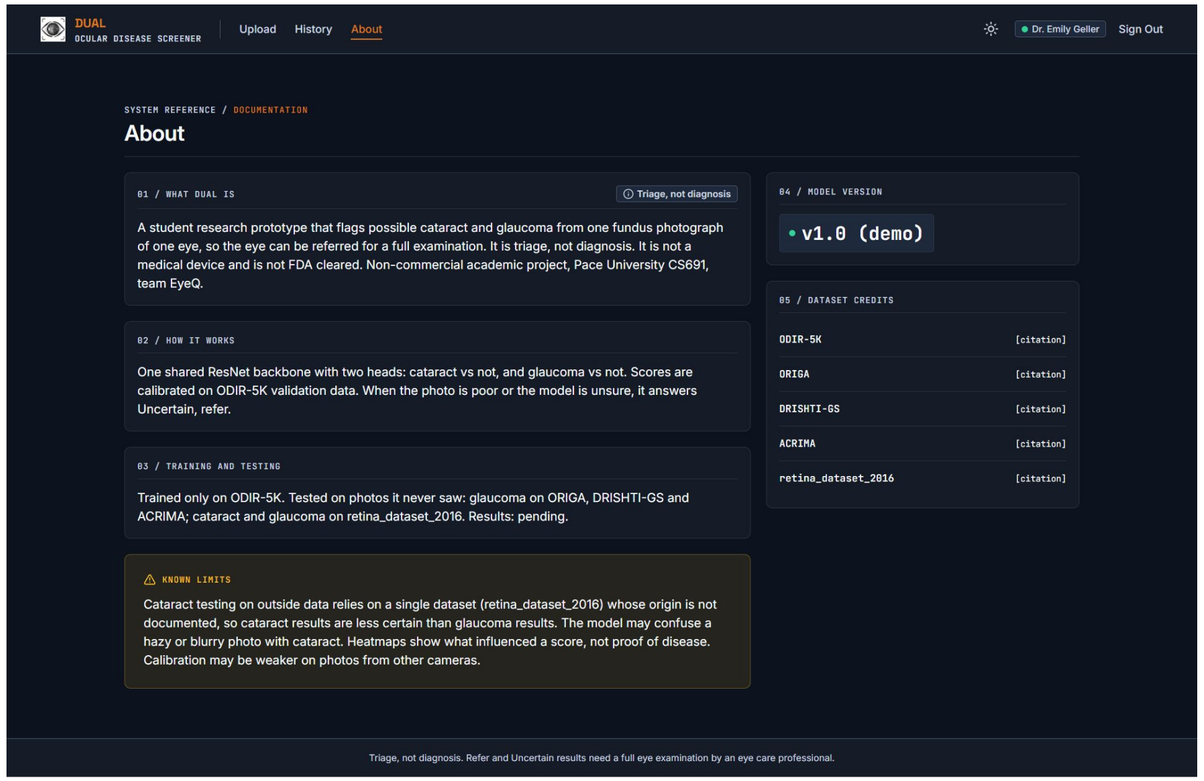
App screenshots: History screen

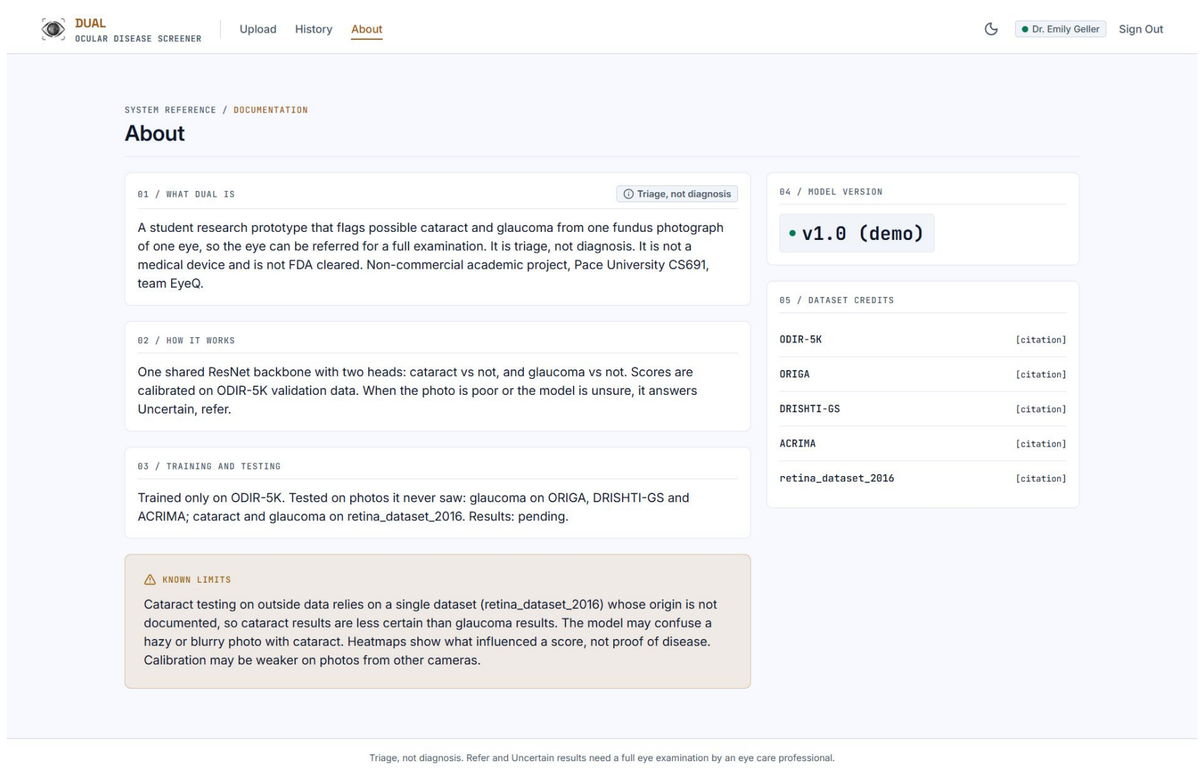




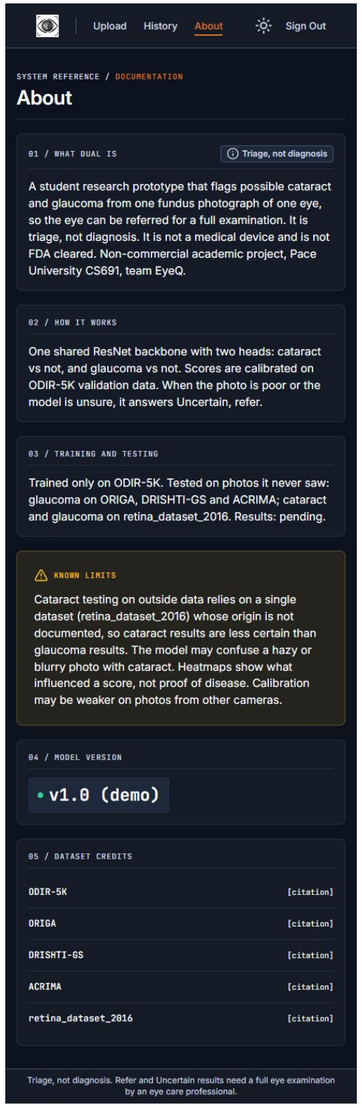
15 • DEMO

App screenshots: About screen

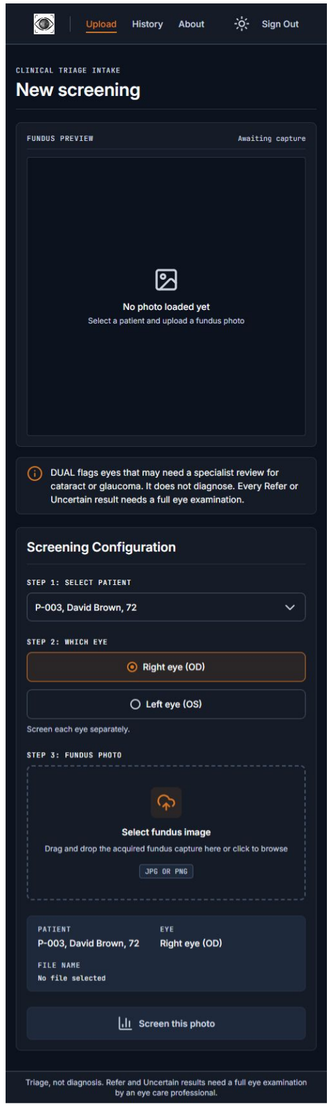


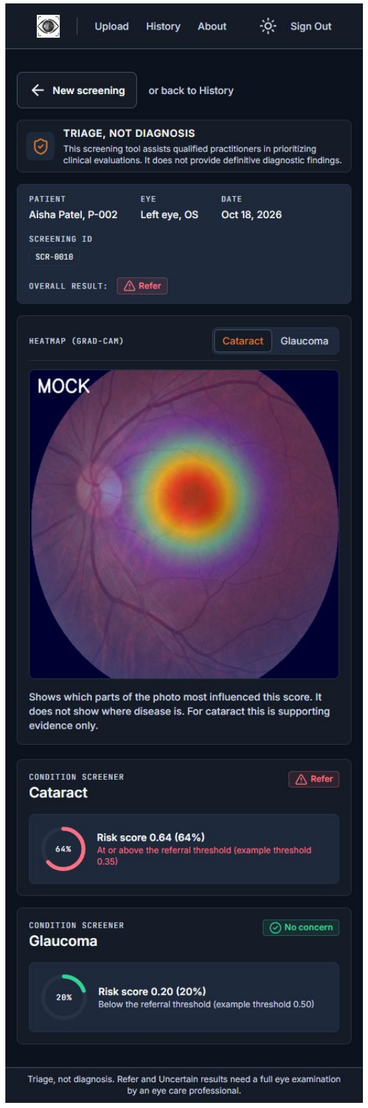


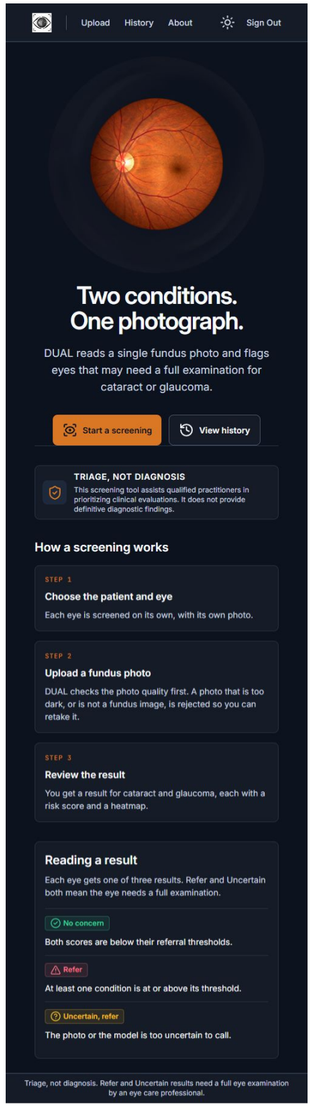
App screenshots: Responsive Design











15 · DEMO

API: what the back end offers

Method	Endpoint	What it does
POST	**/auth/login**	Sign in; sets a session cookie
POST	**/auth/logout**	Sign out
GET	**/auth/me**	Signed-in user
GET	**/patients**	Patients, sorted by last name
POST	**/patients**	Add a patient
GET	**/patients/{id}**	Patient, last visit, latest result per eye
GET	**/patients/{id}/screenings**	A patient's screenings, newest first
GET	**/patients/{id}/export**	Patient dossier (PDF)
GET	**/patients/next-id**	ID for the next new patient

Method	Endpoint	What it does
POST	**/images**	Upload a photo; quality check runs at once
GET	**/images/{id}**	512 x 512 preprocessed photo
GET	**/images/{id}/original**	Photo as uploaded
DELETE	**/images/{id}**	Discard an unscreened photo (retake)
GET	**/heatmaps/{id}**	Heatmap image (mock for now)
POST	**/screenings**	Screen a photo, or skip an eye
GET	**/screenings**	History, newest first
GET	**/screenings/{id}**	Results of one screening
GET	**/screenings/{id}/export**	Screening summary (PDF)
GET	**/model**	Model version and thresholds

FastAPI with Supabase (PostgreSQL and photo storage). The endpoints are unchanged. The API tests were written for the earlier SQLite version; updating them for Supabase is planned for Sprint 2.

15 · DEMO

API: what the back end offers

auth

POST	`/auth/login`	Login
POST	`/auth/logout`	Logout
GET	`/auth/me`	Signed-in user (header name)

patients

GET	`/patients`	Patients, sorted by last name
POST	`/patients`	Add Patient
GET	`/patients/next-id`	ID the next new patient will get
GET	`/patients/{patient_id}`	Patient page: patient, last encounter, latest screening per eye
GET	`/patients/{patient_id}/screenings`	A patient's screenings, newest first (date=today for today's visit)
GET	`/patients/{patient_id}/export`	Export Dossier (PDF)

Sign-in and patients

images

POST	`/images`	Upload a photo; the quality check runs straight away
GET	`/images/{image_id}`	512 x 512 preprocessed photo (JPEG)
DELETE	`/images/{image_id}`	Retake / Upload different photo: discard an unscreened photo
GET	`/images/{image_id}/original`	Photo as uploaded (JPEG)
GET	`/heatmaps/{heatmap_id}`	Heatmap image (JPEG, MOCK for now)

screenings

POST	`/screenings`	Screen this photo, or skip an eye (skipped=true with a skip_reason)
GET	`/screenings`	History, newest first
GET	`/screenings/{screening_id}`	Results and Screening record
GET	`/screenings/{screening_id}/export`	Export summary (PDF)

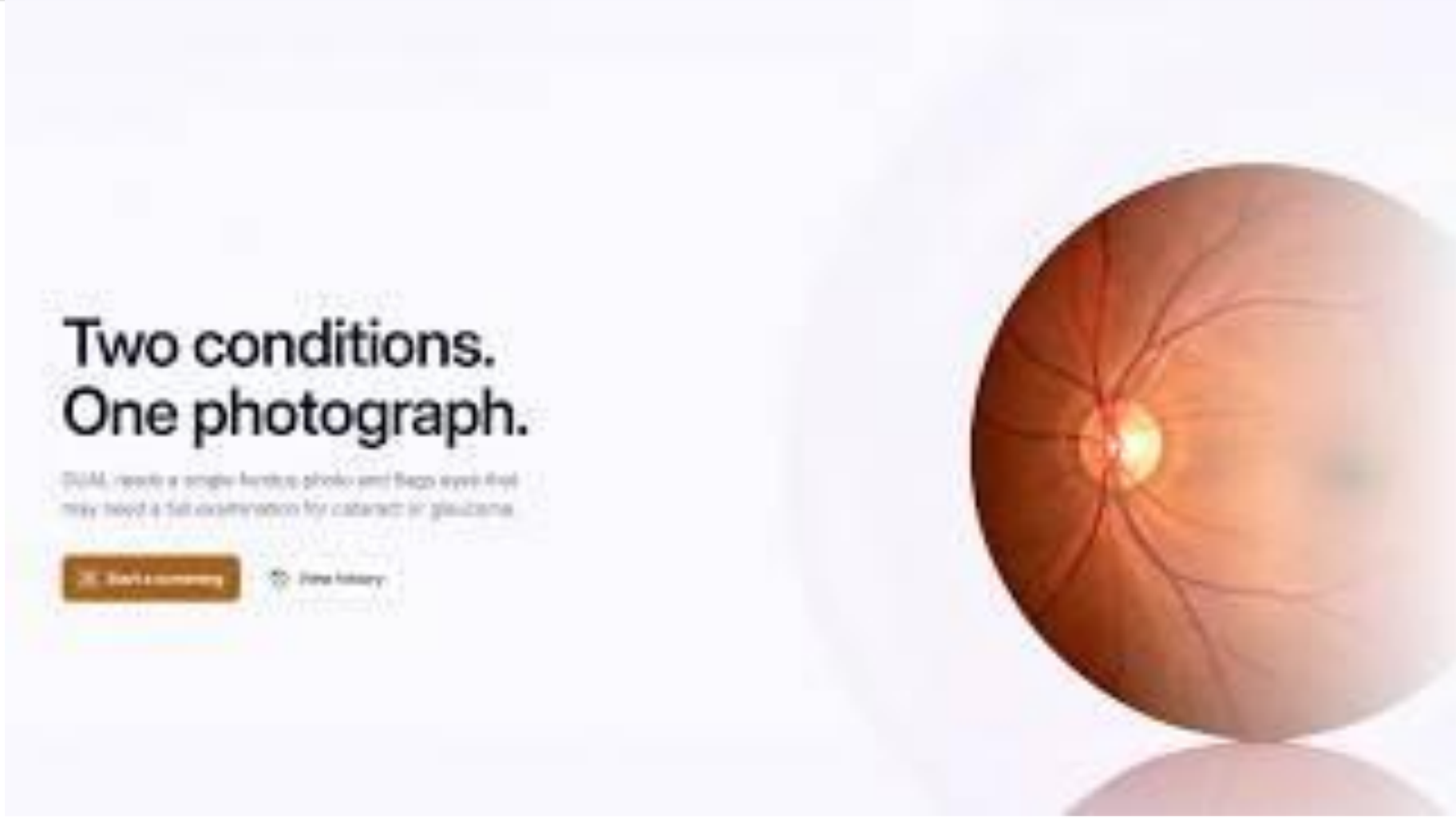
model

GET	`/model`	Current model version and thresholds

Photos, screenings and model

Generated by FastAPI at http://localhost:8000/docs when the back end runs. Each endpoint can be tried from this page.

Project demo



AI DISCLOSURE

Use of AI tools

Tools used

The team used AI assistants as support tools during Sprint 1, as allowed by Professor Wong.

Where it helped

- First, the app design. We used AI to generate the initial screen mockups and our frontend developer used those as references to build the screens in React.
- Secondly, Personas. The persona images shown earlier were AI generated and represent fictitious people.
- Third, project organization. We used AI to check our slides and Wiki against the requirements of the course and to prepare backlog tables and burndown charts using Jira data.

Human review

We reviewed and modified AI assisted work prior to using it with our team. We also validated the project data with Jira and GitHub.

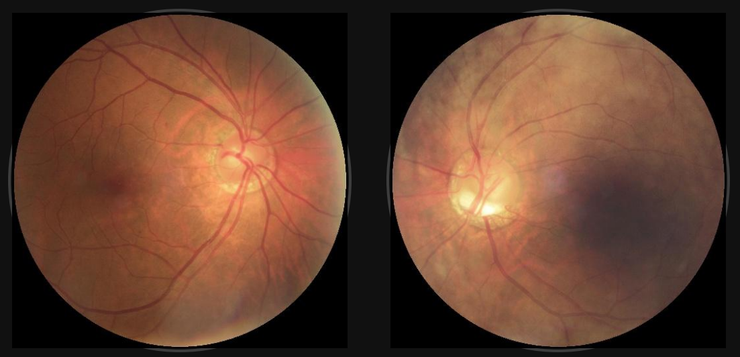
Responsibility

We made all the final decisions, and we are responsible for the code, the documentation and the results we present.

GROUP WIKI

https://github.com/htmw/F2026-EyeQ

Earlier Detection. Brighter Tomorrows.





Thank you

TEAM EYEQ